

CHAPTER 4

Neural Networks based on Competition

NN Based on Competition

- ❑ Specifically, when we applied a net that was trained to classify the input signal into one of the output categories, A, B, C, D, E, J, or K, the net sometimes responded that the signal was both a C and a K, or both an E and a K, or both a J and a K. In circumstances such as this, in which we know that only one of several neurons should respond, we can include additional structure in the network so that the net is forced to make a decision as to which one unit will respond.
- ❑ The mechanism by which this is achieved is called competition.

NN Based on Competition

- ❑ The most extreme form of competition among a group of neurons is called **Winner Take All**.
- ❑ As the name suggests, only one neuron in the competing group will have a nonzero output signal when the competition is completed (**MAXNET**).
- ❑ A more general form of competition is the **Mexican Hat**.

NN Based on Competition

- ☐ Neural network learning is not restricted to supervised learning, wherein training pairs are provided.
- ☐ A second major type of learning for neural networks is unsupervised learning, in which the net seeks to find patterns or regularity in the input data (SOM and ART).
- ☐ In a clustering net, there are as many input units as an input vector has components.
- ☐ Since each output unit represents a cluster, the number of output units will limit the number of clusters that can be formed.
- ☐

NN Based on Competition

- ❑ The weight vector for an output unit in a clustering net (as well as in LVQ nets) serves as a **representative**, or **exemplar**, or **code-book** vector for the input patterns which the net has placed on that cluster.
- ❑ During training, the net determines the output unit that is the best match for the current input vector; the weight vector for the winner is then adjusted in accordance with the net's learning algorithm.

NN Based on Competition

- ❑ Several of the nets discussed in this chapter use the same learning algorithm, known as **Kohonen** learning.
- ❑ the unit whose weight vector was closest to the input vector is allowed to learn:

$$\begin{aligned}w_{.j}(\text{new}) &= w_{.j}(\text{old}) + \alpha[x - w_{.j}(\text{old})] \\ &= \alpha x + (1 - \alpha)w_{.j}(\text{old}).\end{aligned}$$

NN Based on Competition

- ❑ Two methods of determining the closest weight vector to a pattern vector are as follows.
- ❑ The **first** method of determining the winner uses the squared Euclidean distance between the input vector and the weight vector and chooses the unit whose weight vector has the smallest Euclidean distance from the input vector.
- ❑ The **second** method uses the dot product of the input vector and the weight vector.
- ❑ The largest dot product corresponds to the smallest angle between the input and weight vectors if they are both of unit length.
- ❑

NN Based on Competition

- ❑ The dot product can be interpreted as giving the correlation between the input and weight vectors.
- ❑ For vectors of unit length, the two methods (Euclidean and dot product) are equivalent.
- ❑ That is, if the input vectors and the weight vectors are of unit length, the same weight vector will be chosen as closest to the input vector, regardless of whether the Euclidean distance or the dot product method is used.
- ❑ In general, for consistency and to avoid the difficulties of having to normalize our inputs and weights, we shall use the Euclidean distance squared.

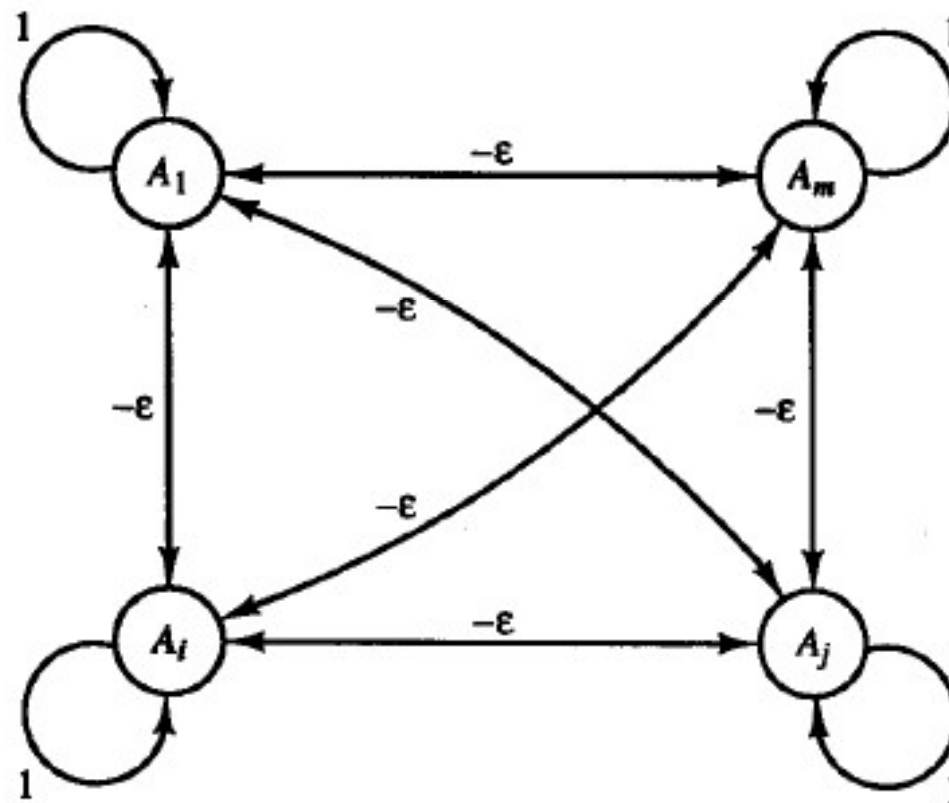
FIXED-WEIGHT NETS

- ❑ Many neural nets use the idea of competition among neurons to enhance the contrast in activations of the neurons.
- ❑ In the most extreme situation, often called Winner-Take-All, only the neuron with the largest activation is allowed to remain "on."

MAXNET

- ❑ MAXNET is a specific example of a neural net based on competition.
- ❑ It can be used as a subnet to pick the node whose input is the largest.
- ❑ The m nodes in this subnet are completely interconnected, with symmetric weights.
- ❑ There is no training algorithm for the MAXNET; the weights are fixed.

MAXNET



Application

- The activation function for the MAXNET is

$$f(x) = \begin{cases} x & \text{if } x \geq 0; \\ 0 & \text{otherwise.} \end{cases}$$

Application

Step 0. Initialize activations and weights (set $0 < \epsilon < \frac{1}{m}$):

$a_j(0)$ input to node A_j ,

$$w_{ij} = \begin{cases} 1 & \text{if } i = j; \\ -\epsilon & \text{if } i \neq j. \end{cases}$$

Step 1. While stopping condition is false, do Steps 2–4.

Step 2. Update the activation of each node: For $j = 1, \dots, m$,

$$a_j(\text{new}) = f[a_j(\text{old}) - \epsilon \sum_{k \neq j} a_k(\text{old})].$$

Step 3. Save activations for use in next iteration:

$$a_j(\text{old}) = a_j(\text{new}), j = 1, \dots, m.$$

Step 4. Test stopping condition:

If more than one node has a nonzero activation, continue;
otherwise, stop.

Example 4.1

- Consider the action of a MAXNET with four neurons and inhibitory weights $\epsilon = 0.2$ when given the initial activations (input signals):

$$a_1(0) = 0.2 \quad a_2(0) = 0.4 \quad a_3(0) = 0.6 \quad a_4(0) = 0.8$$

- The activations found as the net iterates are:

$$a_1(1) = 0.0 \quad a_2(1) = 0.08 \quad a_3(1) = 0.32 \quad a_4(1) = 0.56$$

$$a_1(2) = 0.0 \quad a_2(2) = 0.0 \quad a_3(2) = 0.192 \quad a_4(2) = 0.48$$

$$a_1(3) = 0.0 \quad a_2(3) = 0.0 \quad a_3(3) = 0.096 \quad a_4(3) = 0.442$$

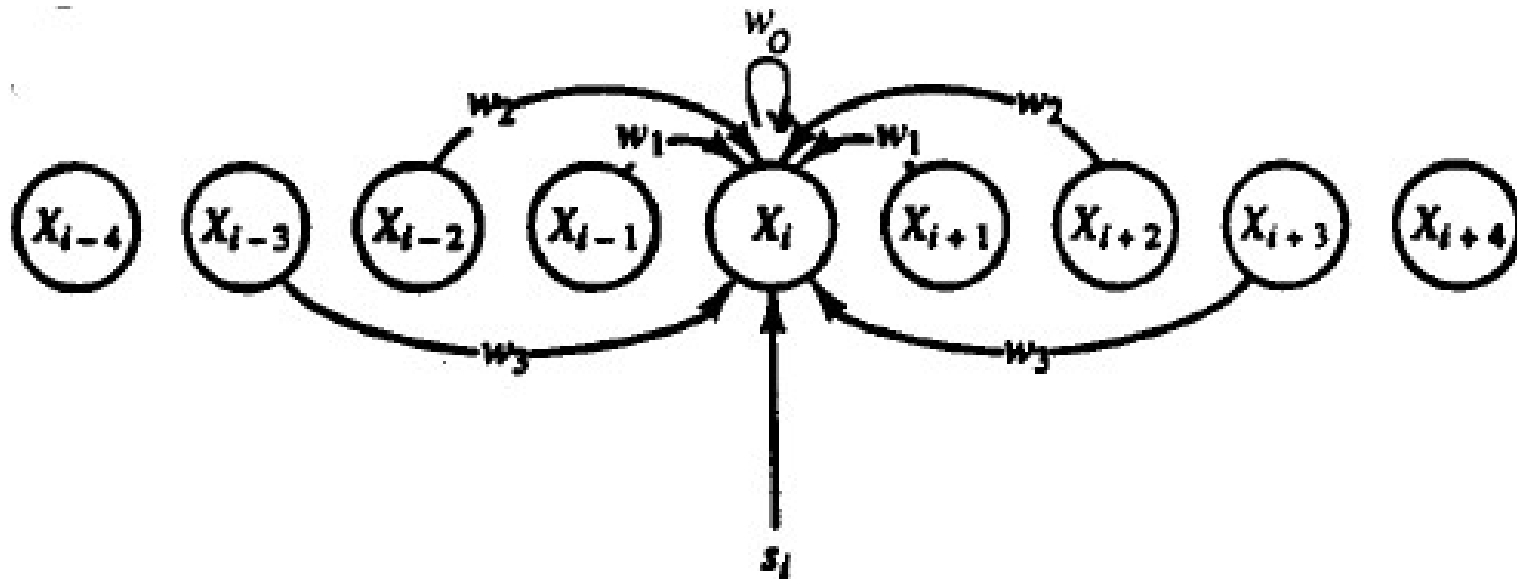
$$a_1(4) = 0.0 \quad a_2(4) = 0.0 \quad a_3(4) = 0.008 \quad a_4(4) = 0.422$$

$$a_1(5) = 0.0 \quad a_2(5) = 0.0 \quad a_3(5) = 0.0 \quad a_4(5) = 0.421$$

Mexican Hat

- ❑ The Mexican Hat network is a more general contrast-enhancing subnet than the MAXNET.
- ❑ Each neuron is connected with excitatory (positively weighted) links to a number of "cooperative neighbors," neurons that are in close proximity.
- ❑ Each neuron is also connected with inhibitory links (with negative weights) to a number of "competitive neighbors," neurons that are somewhat further away.
- ❑ There may also be a number of neurons, further away still, to which the neuron is not connected.

Mexican Hat



Mexican Hat

- ❑ The size of the region of cooperation (positive connections) and the region of competition (negative connections) may vary.
- ❑ The activation of unit X_i at time t is given by:

$$x_i(t) = f[s_i(t) + \sum_k w_k x_{i+k}(t-1)],$$

Algorithm

R_2	Radius of region of interconnections; X_i is connected to units X_{i+k} and X_{i-k} for $k = 1, \dots, R_2$.
R_1	Radius of region with positive reinforcement; $R_1 < R_2$.
w_k	Weight on interconnections between X_i and units X_{i+k} and X_{i-k} : w_k is positive for $0 \leq k \leq R_1$, w_k is negative for $R_1 < k \leq R_2$.
$\mathbf{x}$	Vector of activations.
$\mathbf{x_old}$	Vector of activations at previous time step.
t_max	Total number of iterations of contrast enhancement.
$\mathbf{s}$	External signal.

Algorithm

Step 0. Initialize parameters t_{max} , R_1 , R_2 as desired.
Initialize weights:

$$w_k = C_1 \text{ for } k = 0, \dots, R_1 \text{ } (C_1 > 0)$$

$$w_k = C_2 \text{ for } k = R_1 + 1, \dots, R_2 \text{ } (C_2 < 0).$$

Initialize $\mathbf{x}_{old}$ to 0.

Step 1. Present external signal $\mathbf{s}$:

$$\mathbf{x} = \mathbf{s}.$$

Save activations in array $\mathbf{x}_{old}$ (for $i = 1, \dots, n$):

$$x_{old_i} = x_i.$$

Set iteration counter: $t = 1$.

Algorithm

Step 2. While t is less than t_max , do Steps 3–7.

Step 3. Compute net input ($i = 1, \dots, n$):

$$x_i = C_1 \sum_{k=-R_1}^{R_1} x_old_{i+k} \\ + C_2 \sum_{k=-R_1-1}^{-R_1-1} x_old_{i+k} + C_2 \sum_{k=R_1+1}^{R_2} x_old_{i+k}.$$

Step 4. Apply activation function (ramp function from 0 to x_max , slope 1):

$$x_i = \min(x_max, \max(0, x_i)) \quad (i = 1, \dots, n).$$

Step 5. Save current activations in **x_old**:

$$x_old_i = x_i \quad (i = 1, \dots, n).$$

Step 6. Increment iteration counter:

$$t = t + 1.$$

Step 7. Test stopping condition:

If $t < t_max$, continue; otherwise, stop.

Example 4.2

- ❑ We illustrate the Mexican Hat algorithm for a simple net with seven units.
- ❑ The activation function for this net is:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } 0 \leq x \leq 2 \\ 2 & \text{if } 2 < x. \end{cases}$$

- ❑ Step 0. Initialize parameters:

$$R_1 = 1;$$

$$R_2 = 2;$$

$$C_1 = 0.6;$$

$$C_2 = -0.4.$$

Example 4.2

□ Step 1 . ($t = 0$).

The external signal is (0.0, 0.5, 0.8, 1.0, 0.8, 0.5, 0.0), so

$$\mathbf{x} = (0.0, 0.5, 0.8, 1.0, 0.8, 0.5, 0.0).$$

Save in $\mathbf{x_old}$:

$$\mathbf{x_old} = (0.0, 0.5, 0.8, 1.0, 0.8, 0.5, 0.0).$$

Example 4.2

- Step 2. ($t = 1$). The update formulas used in Step 3 are listed as follows for reference:

$$x_1 = 0.6 x_{\text{old}_1} + 0.6 x_{\text{old}_2} - 0.4 x_{\text{old}_3}$$

$$x_2 = 0.6 x_{\text{old}_1} + 0.6 x_{\text{old}_2} + 0.6 x_{\text{old}_3} - 0.4 x_{\text{old}_4}$$

$$x_3 = -0.4 x_{\text{old}_1} + 0.6 x_{\text{old}_2} + 0.6 x_{\text{old}_3} + 0.6 x_{\text{old}_4} - 0.4 x_{\text{old}_5}$$

$$x_4 = -0.4 x_{\text{old}_2} + 0.6 x_{\text{old}_3} + 0.6 x_{\text{old}_4} + 0.6 x_{\text{old}_5} - 0.4 x_{\text{old}_6}$$

$$x_5 = -0.4 x_{\text{old}_3} + 0.6 x_{\text{old}_4} + 0.6 x_{\text{old}_5} + 0.6 x_{\text{old}_6} - 0.4 x_{\text{old}_7}$$

$$x_6 = -0.4 x_{\text{old}_4} + 0.6 x_{\text{old}_5} + 0.6 x_{\text{old}_6} + 0.6 x_{\text{old}_7}$$

$$x_7 = -0.4 x_{\text{old}_5} + 0.6 x_{\text{old}_6} + 0.6 x_{\text{old}_7}.$$

Example 4.2

□ Step 3. ($t = 1$) .

$$x_1 = 0.6(0.0) + 0.6(0.5) - 0.4(0.8) = -0.2$$

$$x_2 = 0.6(0.0) + 0.6(0.5) + 0.6(0.8) - 0.4(1.0) = 0.38$$

$$x_3 = -0.4(0.0) + 0.6(0.5) + 0.6(0.8) + 0.6(1.0) - 0.4(0.8) = 1.06$$

$$x_4 = -0.4(0.5) + 0.6(0.8) + 0.6(1.0) + 0.6(0.8) - 0.4(0.5) = 1.16$$

$$x_5 = -0.4(0.8) + 0.6(1.0) + 0.6(0.8) + 0.6(0.5) - 0.4(0.0) = 1.06$$

$$x_6 = -0.4(1.0) + 0.6(0.8) + 0.6(0.5) + 0.6(0.0) = 0.38$$

$$x_7 = -0.4(0.8) + 0.6(0.5) + 0.6(0.0) = -0.2.$$

Example 4.2

- ❑ Step 4. $x = (0.0, 0.38, 1.06, 1.16, 1.06, 0.38, 0.0)$.
- ❑ Steps 5-7. Bookkeeping for next iteration.
- ❑ Step 3 ($t=2$)

$$x_1 = 0.6(0.0) + 0.6(0.38) - 0.4(1.06) = -0.196$$

$$x_2 = 0.6(0.0) + 0.6(0.38) + 0.6(1.06) - 0.4(1.16) = 0.39$$

$$x_3 = -0.4(0.0) + 0.6(0.38) + 0.6(1.06) + 0.6(1.16) - 0.4(1.06) = 1.14$$

$$x_4 = -0.4(0.38) + 0.6(1.06) + 0.6(1.16) + 0.6(1.06) - 0.4(0.38) = 1.66$$

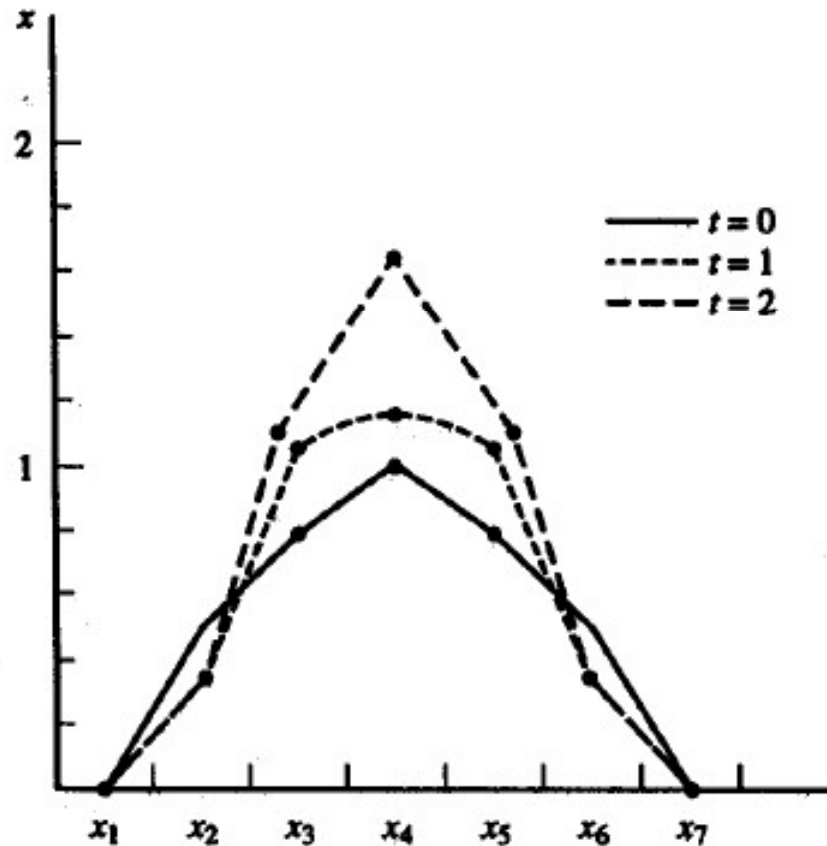
$$x_5 = -0.4(1.06) + 0.6(1.16) + 0.6(1.06) + 0.6(0.38) - 0.4(0.0) = 1.14$$

$$x_6 = -0.4(1.16) + 0.6(1.06) + 0.6(0.38) + 0.6(0.0) = 0.39$$

$$x_7 = -0.4(1.06) + 0.6(0.38) + 0.6(0.0) = -0.196$$

Example 4.2

□ Step 4. $x = (0.0, 0.39, 1.14, 1.66, 1.14, 0.39, 0.0)$.



Hamming Net

- ❑ A Hamming net is a maximum likelihood classifier net that can be used to determine which of several exemplar vectors is most similar to an input vector (an n-tuple).
- ❑ The exemplar vectors determine the weights of the net.
- ❑ The measure of similarity between the input vector and the stored exemplar vectors is n minus the Hamming distance between the vectors.
- ❑ The Hamming distance between two vectors is the number of components in which the vectors differ. For bipolar vectors x and y,

$$\mathbf{x} \cdot \mathbf{y} = a - d,$$

Hamming Net

- where a is the number of components in which the vectors agree and d is the number of components in which the vectors differ, i.e., the Hamming distance.
- However, if n is the number of components in the vectors, then

$$d = n - a$$

- And

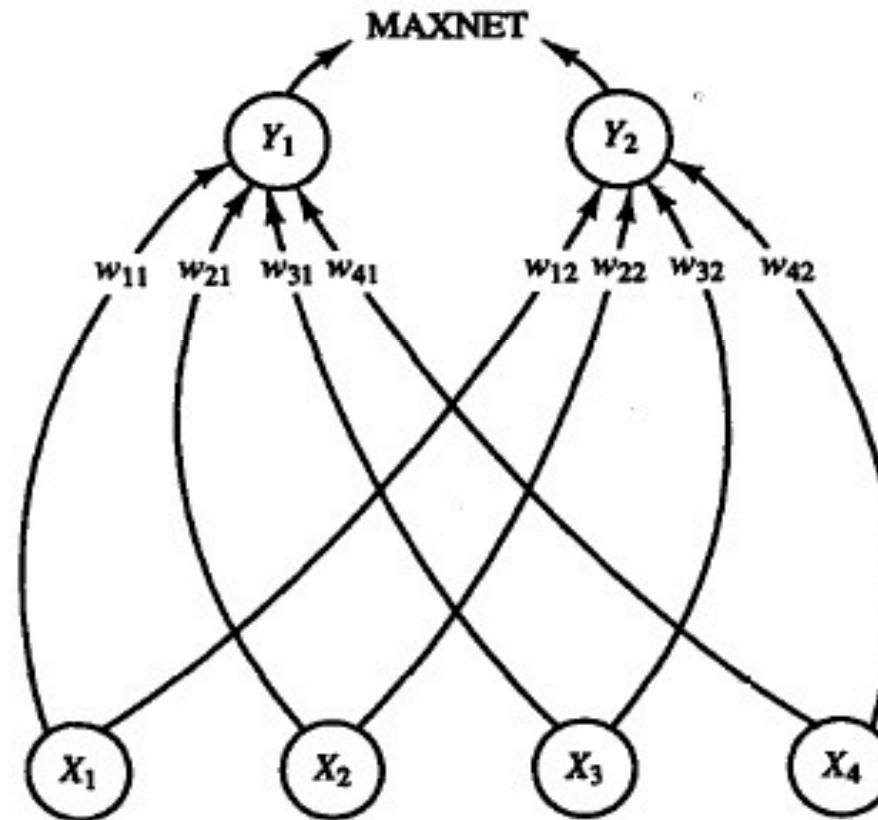
$$\mathbf{x} \cdot \mathbf{y} = 2a - n,$$

- Or

$$2a = \mathbf{x} \cdot \mathbf{y} + n.$$

- By setting the weights to be one-half the exemplar vector and setting the value of the bias to $n/2$, the net will find the unit with the closest exemplar simply by finding the unit with the largest net input.

Architecture



Architecture

- ❑ The Hamming net uses MAXNET as a subnet to find the unit with the largest net input.
- ❑ The lower net consists of n input nodes, each connected to m output nodes (where m is the number of exemplar vectors stored in the net).
- ❑ The output nodes of the lower net feed into an upper net (MAXNET) that calculates the best exemplar match to the input vector.
- ❑ The input and exemplar vectors are bipolar

Application

- Given a set of m bipolar exemplar vectors, $e(1)$, $e(2)$, $\dots$, $e(m)$, the Hamming net can be used to find the exemplar that is closest to the bipolar input vector x .

n number of input nodes, number of components of any input vector;

m number of output nodes, number of exemplar vectors;

$e(j)$ the j th exemplar vector:

$$e(j) = (e_1(j), \dots, e_i(j), \dots, e_n(j)).$$

Application

Step 0. To store the m exemplar vectors, initialize the weights:

$$w_{ij} = \frac{e_i(j)}{2}, (i = 1, \dots, n; j = 1, \dots, m).$$

And initialize the biases:

$$b_j = \frac{n}{2}, (j = 1, \dots, m).$$

Step 1. For each vector $\mathbf{x}$, do Steps 2–4.

Application

Step 2. Compute the net input to each unit Y_j :

$$y_in_j = b_j + \sum_i x_i w_{ij}, (j = 1, \dots, m).$$

Step 3. Initialize activations for MAXNET:

$$y_j(0) = y_in_j, (j = 1, \dots, m).$$

Step 4. MAXNET iterates to find the best match exemplar.

Example 4.3

- ❑ Hamming net to cluster four vectors.
- ❑ Given the exemplar vectors:

$$\mathbf{e}(1) = (1, -1, -1, -1)$$

$$\mathbf{e}(2) = (-1, -1, -1, 1),$$

- ❑ the Hamming net can be used to find the exemplar that is closest to each of the bipolar input patterns, $(1, 1, -1, -1)$, $(1, -1, -1, -1)$, $(-1, -1, -1, 1)$, and $(-1, -1, 1, 1)$.
- ❑ Step 0. Store the m exemplar vectors in the weights:

Example 4.3

$$W = \begin{bmatrix} .5 & -.5 \\ -.5 & -.5 \\ -.5 & -.5 \\ -.5 & .5 \end{bmatrix}.$$

Initialize the biases:

$$b_1 = b_2 = 2.$$

- ❑ Step 1, For the vector $x = (1, 1, -1, -1)$, do Steps 2-4.

- ❑ Step 2.

$$\text{Step 2.} \quad y_{\text{in}_1} = b_1 + \sum_i x_i w_{i1}$$

$$= 2 + 1 = 3;$$

$$y_{\text{in}_2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 - 1 = 1.$$

Example 4.3

- ❑ These values represent the Hamming similarity because $(1, 1, -1, -1)$ agrees with $e(1) = (1, -1, -1, -1)$ in the first, third, and fourth components and because $(1, 1, -1, -1)$ agrees with $e(2) = (-1, -1, -1, 1)$ in only the third component.
- ❑ Step 3. $y_1(0) = 3;$
 $y_2(0) = 1.$
- ❑ Step 4. Since $y_1(0) > y_2(0)$, MAXNET will find that unit Y1 has the best match exemplar for input vector $x = (1, 1, -1, -1)$.

Example 4.3

- ❑ Step 1 . For the vector $x = (1, -1, -1, -1)$. do Steps

$$2-4. \quad y_{in1} = b_1 + \sum_i x_i w_{i1}$$

$$= 2 + 2 = 4;$$

$$y_{in2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 + 0 = 2.$$

- ❑ Note that the input vector agrees with $e(1)$ in all four components and agrees with $e(2)$ in the second and third components.

- ❑ Step 3.

$$y_1(0) = 4;$$

$$y_2(0) = 2.$$

Example 4.3

- ❑ Step 4. Since $y_1(0) > y_2(0)$, MAXNET will find that unit Y1 has the best match exemplar for input vector $x = (1, -1, -1, -1)$.
- ❑ Step 1. For the vector $x = (-1, -1, -1, 1)$, do Steps 2-4.

- ❑ Step 2.

$$y_{in_1} = b_1 + \sum_i x_i w_{i1}$$

$$= 2 + 0 = 2;$$

$$y_{in_2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 + 2 = 4.$$

Example 4.3

- ❑ The input vector agrees with $e(1)$ in the second and third components and agrees with $e(2)$ in all four components.
- ❑ Step 3. $y_1(0) = 2$;
 $y_2(0) = 4$.
- ❑ Step 4. Since $y_2(0) > y_1(0)$, MAXNET will find that unit Y2 has the best match exemplar for input vector $x = (-1, -1, -1, 1)$.

Example 4.3

- Step 1. For the vector $x = (-1, -1, 1, 1)$, do Steps 2-4.

$$\begin{aligned}y_{in_1} &= b_1 + \sum_i x_i w_{i1} \\&= 2 - 1 = 1; \\y_{in_2} &= b_2 + \sum_i x_i w_{i2} \\&= 2 + 1 = 3.\end{aligned}$$

- The input vector agrees with $e(1)$ in the second component and agrees with $e(2)$ in the first, second, and fourth components.

Example 4.3

❑ Step 3.

$$y_1(0) = 1;$$

$$y_2(0) = 3.$$

❑ Step 4. Since $y_2(0) > y_1(0)$, MAXNET will find that unit Y2 has the best match exemplar for input vector $x = (-1, -1, 1, 1)$.

KOHONEN SOM

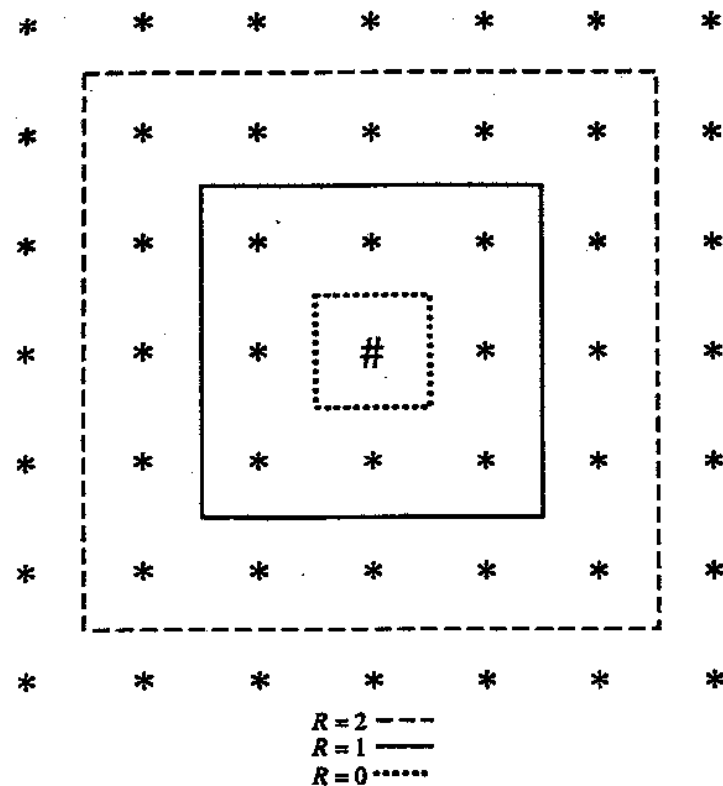
- ❑ The self-organizing neural networks described in this section, also called topology preserving maps, assume a topological structure among the cluster units.
- ❑ This property is observed in the brain, but is not found in other artificial neural networks.
- ❑ There are m cluster units, arranged in a one- or two-dimensional array; the input signals are n -tuples.

KOHONEN SOM

- ❑ The weight vector for a cluster unit serves as an exemplar of the input patterns associated with that cluster.
- ❑ During the self-organization process, the cluster unit whose weight vector matches the input pattern most closely (typically, the square of the minimum Euclidean distance) is chosen as the winner.
- ❑ The winning unit and its neighboring units (in terms of the topology of the cluster units) update their weights.

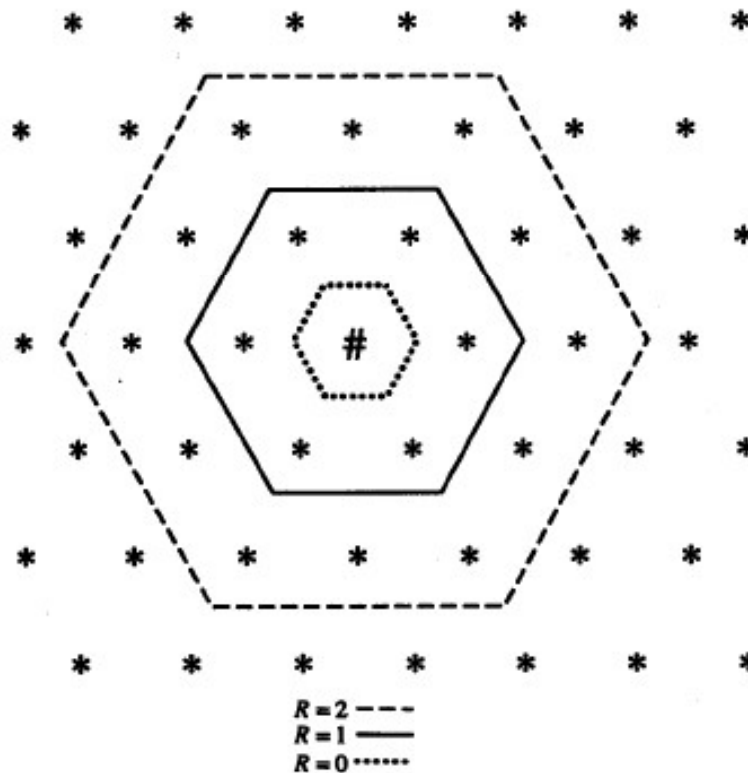
Architecture

- rectangular grid.



Architecture

- ❑ hexagonal grid.



Linear array of cluster units

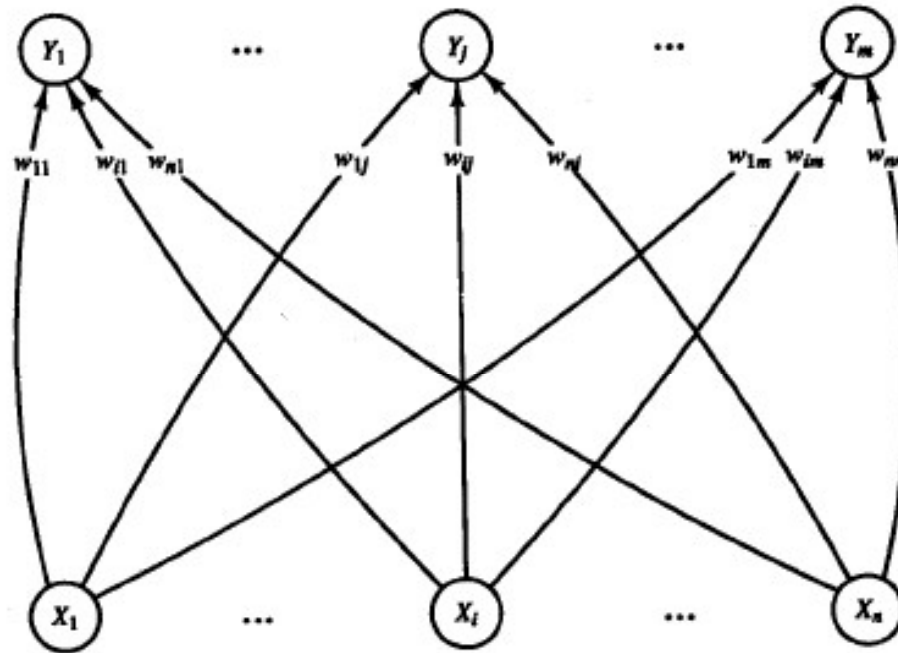


Figure 4.5 Kohonen self-organizing map.

* * * { * (* [#] *) * } * *

{ } R=2 () R=1 [] R=0

Algorithm

4.2.2 Algorithm

Step 0. Initialize weights w_{ij} . (Possible choices are discussed below.)
Set topological neighborhood parameters.
Set learning rate parameters.

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each input vector $\mathbf{x}$, do Steps 3–5.

Step 3. For each j , compute:

$$D(j) = \sum_i (w_{ij} - x_i)^2.$$

Step 4. Find index J such that $D(J)$ is a minimum.

Step 5. For all units j within a specified neighborhood of J , and for all i :

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})].$$

Step 6. Update learning rate.

Algorithm

Step 7. Reduce radius of topological neighborhood at specified times.

Step 8. Test stopping condition.

- ☐ Alternative structures are possible for reducing R and learning rate.
- ☐ The learning rate is a slowly decreasing function of time (or training epochs).

Algorithm

- ❑ The radius of the neighborhood around a cluster unit also decreases as the clustering process progresses.
- ❑ The formation of a map occurs in two phases: the initial formation of the correct order and the final convergence.
- ❑ The second phase takes much longer than the first and requires a small value for the learning rate.
- ❑ Many iterations through the training set may be necessary, at least in some applications.

Example 4.4

- ❑ A Kohonen self-organizing map (SOM) to cluster four vectors.

- ❑ Let the vectors to be clustered be:

$(1, 1, 0, 0); (0, 0, 0, 1); (1, 0, 0, 0); (0, 0, 1, 1).$

- ❑ The maximum number of clusters to be formed is

$$m = 2.$$

- ❑ Suppose the learning rate (geometric decrease) is:

$$\alpha(0) = .6,$$

$$\alpha(t + 1) = .5 \alpha(t).$$

Example 4.4

- ❑ With only two clusters available, the neighborhood of node J (Step 4) is set so that only one cluster updates its weights at each step (i.e., $R = 0$).
- ❑ Step 0. Initial weight matrix:
$$\begin{bmatrix} .2 & .8 \\ .6 & .4 \\ .5 & .7 \\ .9 & .3 \end{bmatrix}.$$
- ❑ Initial radius: $R=0$.
- ❑ Initial learning rate: $\alpha(0) = 0.6$.
- ❑ Step 1. Begin training.
- ❑ Step 2. For the first vector, (1 , 1 , 0, 0), do Steps 3-5.

Example 4.4

❑ Step 3.

$$D(1) = (.2 - 1)^2 + (.6 - 1)^2 + (.5 - 0)^2 + (.9 - 0)^2 = 1.86;$$
$$D(2) = (.8 - 1)^2 + (.4 - 1)^2 + (.7 - 0)^2 + (.3 - 0)^2 = 0.98.$$

- ❑ Step 4. The input vector is closest to output node 2, so $J = 2$.
- ❑ Step 5. The weights on the winning unit are updated:

$$\begin{aligned}w_{J2}(\text{new}) &= w_{J2}(\text{old}) + .6 [x_i - w_{J2}(\text{old})] \\&= .4 w_{J2}(\text{old}) + .6 x_i.\end{aligned}$$

Example 4.4

- ❑ This gives the weight matrix $\begin{bmatrix} .2 & .92 \\ .6 & .76 \\ .5 & .28 \\ .9 & .12 \end{bmatrix}$
- ❑ Step 2. For the second vector, (0, 0, 0, 1), do Steps 3-5.
- ❑ Step 3.
$$D(1) = (.2 - 0)^2 + (.6 - 0)^2 + (.5 - 0)^2 + (.9 - 1)^2 = 0.66;$$
$$D(2) = (.92 - 0)^2 + (.76 - 0)^2 + (.28 - 0)^2 + (.12 - 1)^2 = 2.2768.$$

Example 4.4

- ❑ Step 4. The input vector is closest to output node 1 ,
so
- ❑ $J=1$.
- ❑ Step 5. Update the first column of the weight matrix:

$$\begin{bmatrix} .08 & .92 \\ .24 & .76 \\ .20 & .28 \\ .96 & .12 \end{bmatrix}$$

Example 4.4

- ❑ Step 2. For the third vector, (1 , 0, 0, 0), do Steps 3-5.

$$D(1) = (.08 - 1)^2 + (.24 - 0)^2 \\ + (.2 - 0)^2 + (.96 - 0)^2 = 1.8656;$$

- ❑ Step 3.

$$D(2) = (.92 - 1)^2 + (.76 - 0)^2 \\ + (.28 - 0)^2 + (.12 - 0)^2 = 0.6768.$$

- ❑ Step 4. The input vector is closest to output node 2, so

- ❑ J = 2.

$$\begin{bmatrix} .08 & .968 \\ .24 & .304 \\ .20 & .112 \\ .96 & .048 \end{bmatrix}$$

Example 4.4

- ❑ Step 2. For the fourth vector, (0 , 0, 1, 1), do Steps 3-5.

- ❑ Step 3.
$$D(1) = (.08 - 0)^2 + (.24 - 0)^2 + (.2 - 1)^2 + (.96 - 1)^2 = 0.7056;$$

$$D(2) = (.968 - 0)^2 + (.304 - 0)^2 + (.112 - 1)^2 + (.048 - 1)^2 = 2.724.$$

- ❑ Step 4. The input vector is closest to output node 1, so

- ❑ $J = 1.$

$$\begin{bmatrix} .032 & .968 \\ .096 & .304 \\ .680 & .112 \\ .984 & .048 \end{bmatrix}$$

Example 4.4

- ❑ Step 6. Reduce the learning rate:

$$\alpha = .5 (0.6) = .3$$

- ❑ The weight update equations are now:

$$\begin{aligned} w_{ij}(\text{new}) &= w_{ij}(\text{old}) + .3 [x_i - w_{ij}(\text{old})] \\ &= .7w_{ij}(\text{old}) + .3x_i. \end{aligned}$$

- ❑ Modifying the adjustment procedure for the learning rate so that it decreases geometrically from .6 to .01 over 100 iterations (epochs) gives the following results:

Example 4.4

Iteration 2: Weight matrix:

$$\begin{bmatrix} .0053 & .9900 \\ -.1700 & .3000 \\ .7000 & .0200 \\ 1.0000 & .0086 \end{bmatrix}$$

Iteration 10: Weight matrix:

$$\begin{bmatrix} 1.5e-7 & 1.0000 \\ 4.6e-7 & .3700 \\ .6300 & 5.4e-7 \\ 1.0000 & 2.3e-7 \end{bmatrix}$$

Iteration 50: Weight matrix:

$$\begin{bmatrix} 1.9e-19 & 1.0000 \\ 5.7e-15 & .4700 \\ .5300 & 6.6e-15 \\ 1.0000 & 2.8e-15 \end{bmatrix}$$

Iteration 100: Weight matrix:

$$\begin{bmatrix} 6.7e-17 & 1.0000 \\ 2.0e-16 & .4900 \\ .5100 & 2.3e-16 \\ 1.0000 & 1.0e-16 \end{bmatrix}$$

Example 4.4

- ❑ These weight matrices appear to be converging to the matrix

$$\begin{bmatrix} 0.0 & 1.0 \\ 0.0 & 0.5 \\ 0.5 & 0.0 \\ 1.0 & 0.0 \end{bmatrix}$$

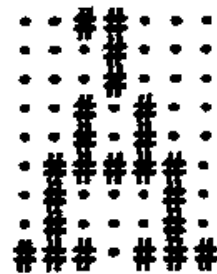
- ❑ the first column of which is the average of the two vectors placed in cluster 1 and the second column of which is the average of the two vectors placed in cluster 2.

Character Recognition

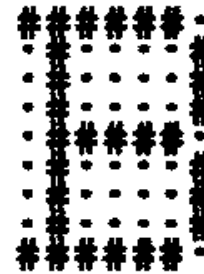
- ❑ Examples 4.5-4.7 show typical results from using a Kohonen self-organizing map to cluster input patterns representing letters in three different fonts.
- ❑ The input patterns for fonts 1, 2, and 3 are given in Figure 4.9.
- ❑ In each of the examples, 25 cluster units are available, which means that a maximum of 25 clusters may be formed.

Training patterns

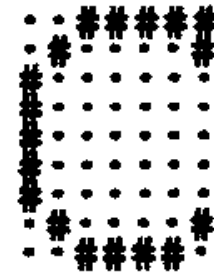
Input from
Font 1



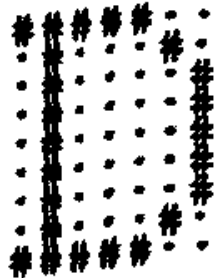
A1



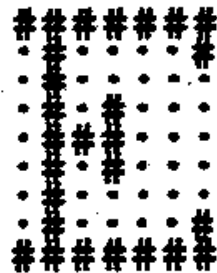
B1



C1



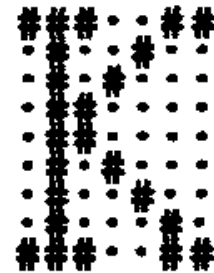
D1



E1

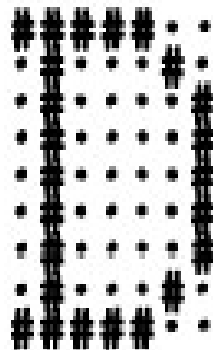


J1

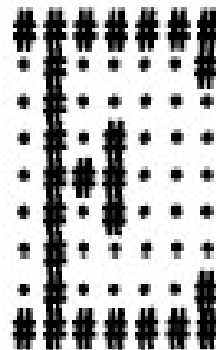


K1

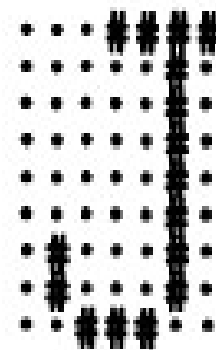
Training patterns



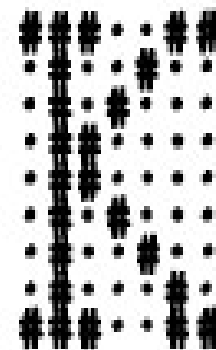
D1



E1

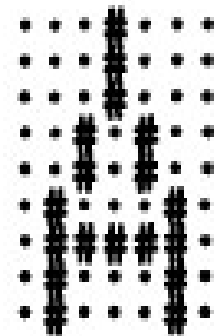


J1

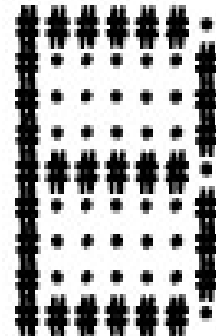


K1

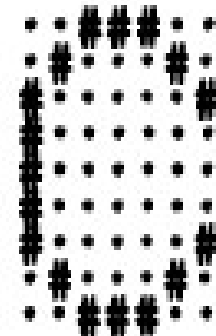
Input from
Font 2



A2



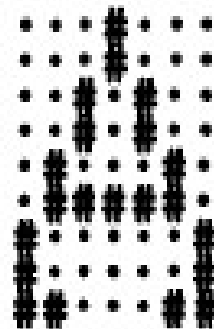
B2



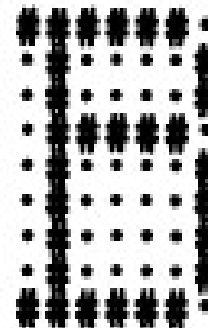
C2

Training patterns

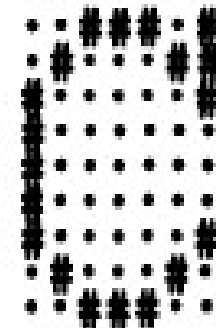
Input from
Font 3



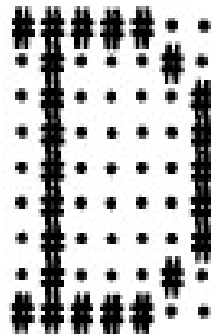
A3



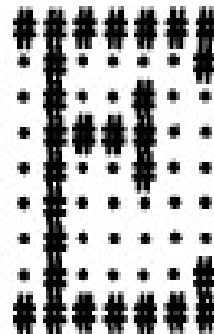
B3



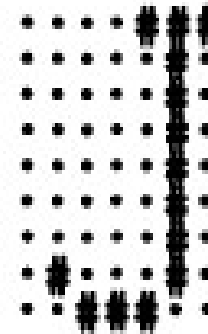
C3



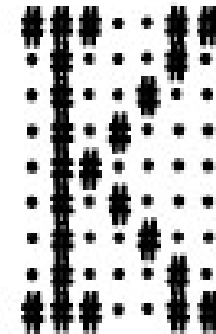
D3



E3



J3



K3

Example 4.5

- ❑ A SOM to cluster letters from different fonts: no topological structure.
- ❑ If no structure is assumed for the cluster units, i.e., if only the winning unit is allowed to learn the pattern presented, the 21 patterns form 5 clusters:

UNIT	PATTERNS
3	C1, C2, C3
13	B1, B3, D1, D3, E1, K1, K3
16	A1, A2, A3
18	J1, J2, J3
24	B2, D2, E2, K2

Example 4.6

- A linear structure (with $R = 1$) gives a better distribution of the patterns onto the available cluster units. The winning node J and its topological neighbors ($J + 1$ and $J - 1$) are allowed to learn on each iteration.

UNIT	PATTERNS	UNIT	PATTERNS
6	K2	20	C1, C2, C3
10	J1, J2, J3	22	D2
14	E1, E3	23	B2, E2
16	K1, K3	25	A1, A2, A3
18	B1, B3, D1, D3		

Example 4.7

- ❑ A SOM to cluster letters from different fonts: diamond structure.
- ❑ In this example, a simple two-dimensional topology is assumed for the cluster units, so that each cluster unit is indexed by two subscripts.
- ❑ If unit X_{IJ} is the winning unit, the units $X_{I+1, J}$; $X_{I-1, J}$; $X_{I, J+1}$, and $X_{I, J-1}$ also learn.

Example 4.7

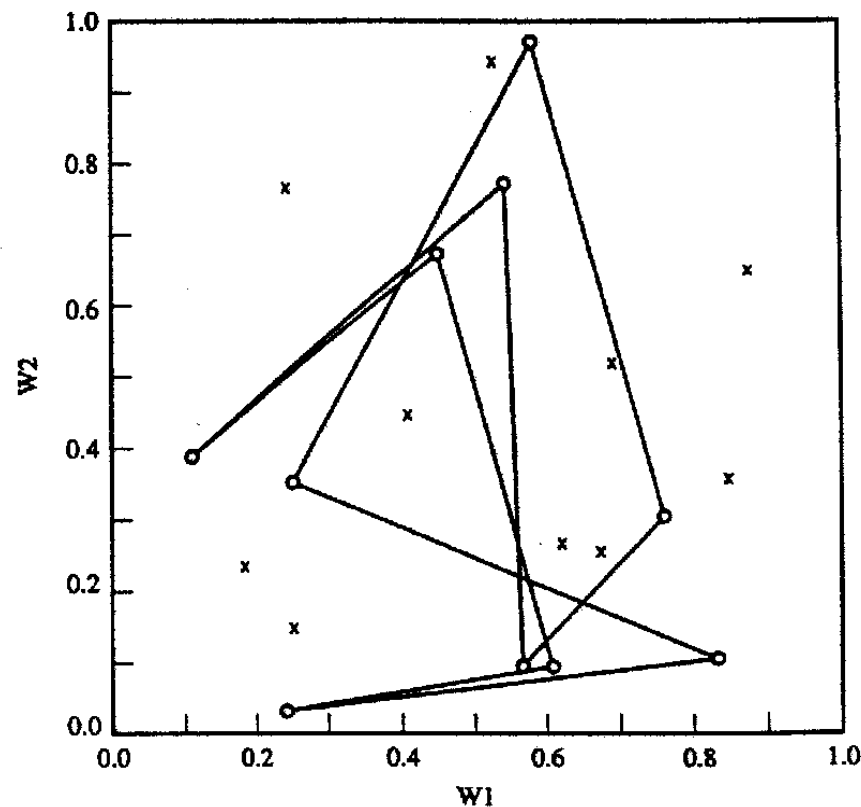
$i \backslash j$	1	2	3	4	5
1		J1, J2, J3		D2	
2	C1, C2, C3		D1, D3		B2, E2
3		B1		K2	
4			E1, E3, B3		A3
5		K1, K3		A1, A2	

Example 4.10

- ❑ Using a SOM: The Traveling Salesman Problem.
- ❑ However, the results can easily be interpreted as representing one of the tours
- ❑ A D E F G H I J B C
- ❑ and
- ❑ A D E F G H I J C B .
- ❑ The same tour (with the same ambiguity) was found, using a variety of initial weights.

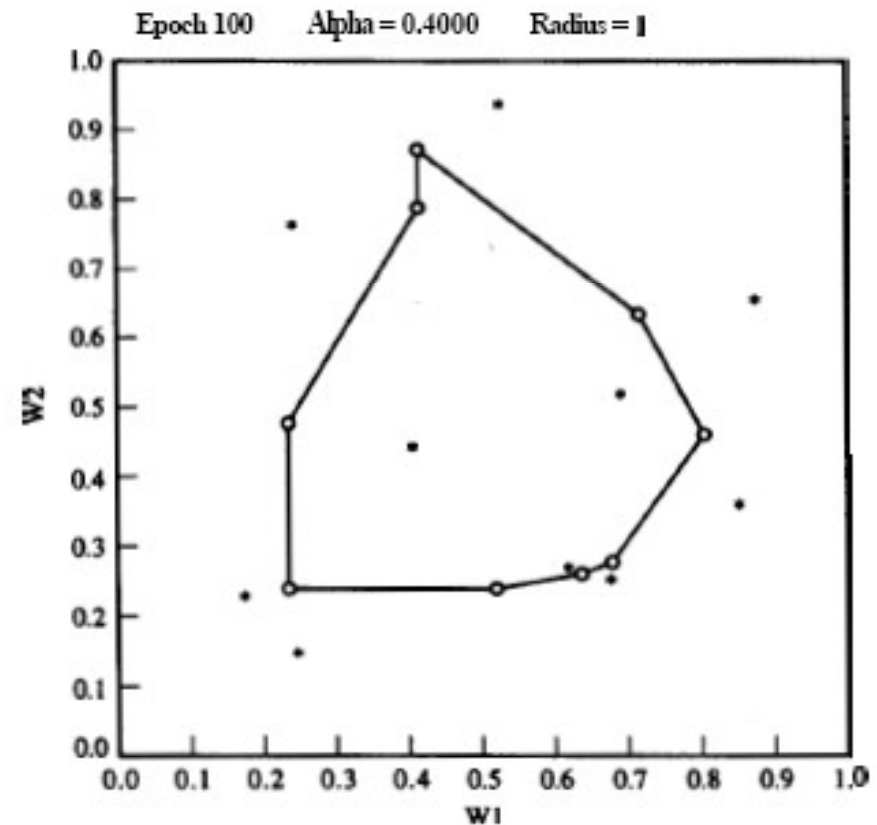
Example 4.10

- Initial position of cluster units and location of cities.



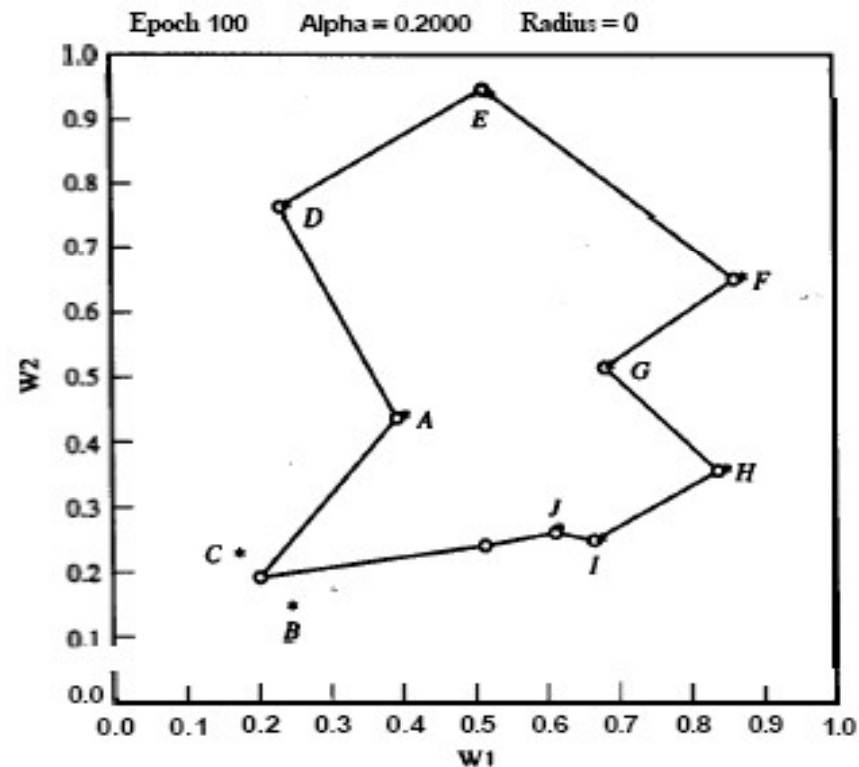
Example 4.10

- ❑ Position of cluster units and location of cities after 100 epochs with $R = 1..$



Example 4.10

- Position of cluster units and location of cities after additional 100 epochs with $R = 0$.

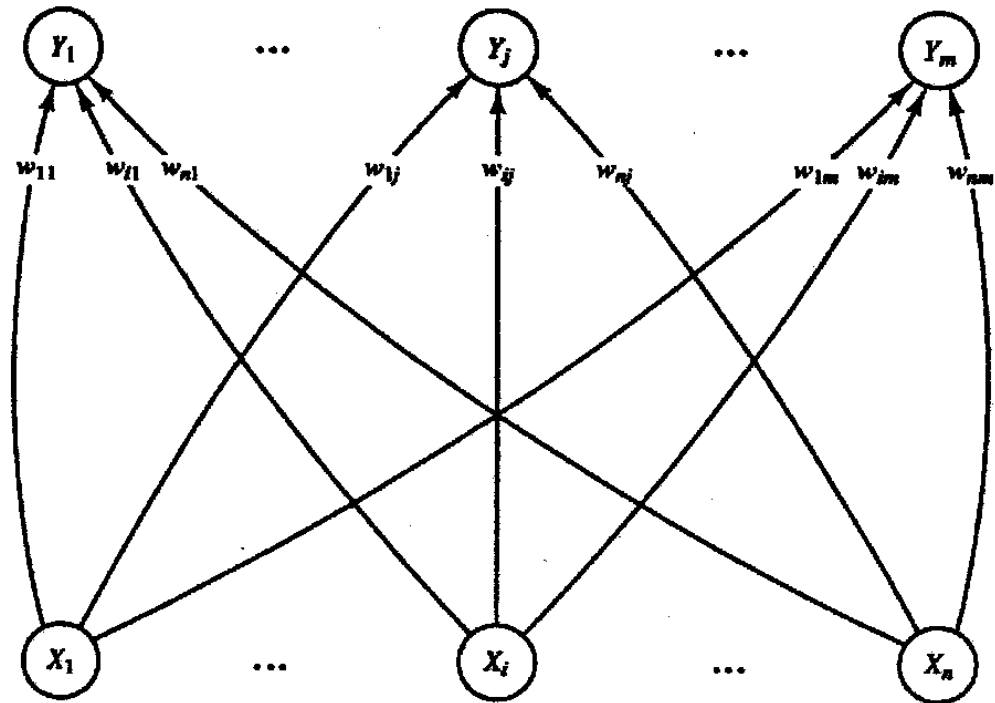


LVQ

- ❑ Learning vector quantization (LVQ) is a pattern classification method in which each output unit represents a particular class or category.
- ❑ The weight vector for an output unit is often referred to as a reference (or codebook) vector for the class that the unit represents.
- ❑ During training, the output units are positioned to approximate the decision surfaces of the theoretical Bayes classifier.
- ❑ After training, an LVQ net classifies an input vector by assigning it to the same class as the output unit that has its weight vector (reference vector) closest to the input vector

Architecture

- ❑ The architecture of an LVQ neural net, is essentially the same as that of a Kohonen self-organizing map (without a topological structure being assumed for the output units)



Algorithm

- ❑ The motivation for the algorithm for the LVQ net is to find the output unit that is closest to the input vector.
- ❑ Toward that end, if x and w_j belong to the same class, then we move the weights toward the new input vector; if x and w_j belong to different classes, then we move the weights away from this input vector.

x	training vector $(x_1, \dots, x_i, \dots, x_n)$.
T	correct category or class for the training vector.
w_j	weight vector for j th output unit $(w_{1j}, \dots, w_{ij}, \dots, w_{nj})$.
C_j	category or class represented by j th output unit.
$\ x - w_j\ $	Euclidean distance between input vector and (weight vector for) j th output unit.

Algorithm

- Step 0.* Initialize reference vectors (several strategies are discussed shortly); initialize learning rate, $\alpha(0)$.
- Step 1.* While stopping condition is false, do Steps 2–6.
- Step 2.* For each training input vector $\mathbf{x}$, do Steps 3–4.
- Step 3.* Find J so that $\|\mathbf{x} - \mathbf{w}_J\|$ is a minimum.
- Step 4.* Update $\mathbf{w}_J$ as follows:
if $T = C_J$, then
$$\mathbf{w}_J(\text{new}) = \mathbf{w}_J(\text{old}) + \alpha[\mathbf{x} - \mathbf{w}_J(\text{old})];$$

if $T \neq C_J$, then
$$\mathbf{w}_J(\text{new}) = \mathbf{w}_J(\text{old}) - \alpha[\mathbf{x} - \mathbf{w}_J(\text{old})].$$
- Step 5.* Reduce learning rate.
- Step 6.* Test stopping condition:
The condition may specify a fixed number of iterations (i.e., executions of Step 1) or the learning rate reaching a sufficiently small value.

Application

- ❑ The simplest method of initializing the weight (reference) vectors is to take the first m training vectors and use them as weight vectors; the remaining vectors are then used for training (Example 4.11).
- ❑ Another simple method, is to assign the initial weights and classifications randomly. (Example 4.12).
- ❑ Another possible method of initializing the weights is to use K-means clustering or the self-organizing map to place the weights.

Example 4.11

- ❑ Learning vector quantization (LVQ): five vectors assigned to two classes.
- ❑ The following input vectors represent two classes, 1 and 2:

VECTOR	CLASS
(1, 1, 0, 0)	1
(0, 0, 0, 1)	2
(0, 0, 1, 1)	2
(1, 0, 0, 0)	1
(0, 1, 1, 0)	2

- ❑ The first two vectors will be used to initialize the two reference vectors.
- ❑ Thus, the first output unit represents class 1, the second class 2 (symbolically, $C_1 = 1$ and $C_2 = 2$).

Example 4.11

- ❑ This leaves vectors (0, 0, 1, 1), (1, 0, 0, 0), and (0, 1, 1, 0) as the training vectors.
- ❑ Only one iteration (one epoch) is shown:
- ❑ Step 0. Initialize weights:
 - $W1 = (1, 1, 0, 0)$;
 - $W2 = (0, 0, 0, 1)$.
 - Initialize the learning rate: $\alpha = .1$

Example 4.11

Step 1. Begin computations.

Step 2. For input vector $\mathbf{x} = (0, 0, 1, 1)$ with $T = 2$, do Steps 3–4.

Step 3. $J = 2$, since $\mathbf{x}$ is closer to $\mathbf{w}_2$ than to $\mathbf{w}_1$.

Step 4. Since $T = 2$ and $C_2 = 2$, update $\mathbf{w}_2$ as follows:

$$\begin{aligned}\mathbf{w}_2 &= (0, 0, 0, 1) + .1 [(0, 0, 1, 1) - (0, 0, 0, 1)] \\ &= (0, 0, .1, 1).\end{aligned}$$

Step 2. For input vector $\mathbf{x} = (1, 0, 0, 0)$ with $T = 1$, do Steps 3–4.

Step 3. $J = 1$.

Step 4. Since $T = 1$ and $C_1 = 1$, update $\mathbf{w}_1$ as follows:

$$\begin{aligned}\mathbf{w}_1 &= (1, 1, 0, 0) + .1 [(1, 0, 0, 0) - (1, 1, 0, 0)] \\ &= (1, .9, 0, 0).\end{aligned}$$

Example 4.11

Step 2. For input vector $\mathbf{x} = (0, 1, 1, 0)$ with $T = 2$, do Steps 3–4.

Step 3. $J = 1$.

Step 4. Since $T = 2$, but $C_1 = 1$, update $\mathbf{w}_1$ as follows:

$$\begin{aligned}\mathbf{w}_1 &= (1, .9, 0, 0) - .1[(0, 1, 1, 0) - (1, .9, 0, 0)] \\ &= (1.1, .89, -.1, 0).\end{aligned}$$

Step 5. This completes one epoch of training.
Reduce the learning rate.

Step 6. Test the stopping condition.

Example 4.12

- ❑ Using LVQ: a geometric example with four cluster units.
- ❑ This example shows the use of LVQ to represent points in the unit square as belonging to one of four classes, indicated by the symbols +, 0, #, and @.
- ❑ There are four cluster units, one for each class.

» INITIAL WEIGHTS

❑ Class 1(+)	0	0
❑ Class 2 (0)	1	1
❑ Class 3 (*)	1	0
❑ Class 4 (#)	0	1

Example 4.12

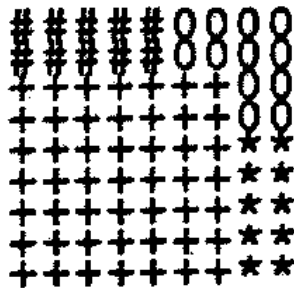


Figure 4.28
Training data

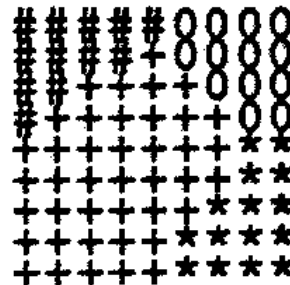


Figure 4.29
Results after one epoch.

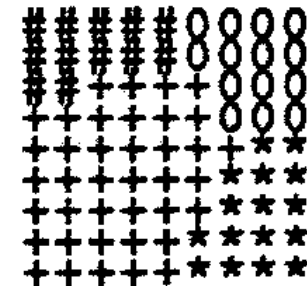


Figure 4.30
Results after two epochs.

Initial Weights:			Weights after epoch 1:			Weights after epoch 2:		
"+"	0.00	0.00	"+"	0.44	0.52	"+"	0.41	0.55
"0"	1.00	1.00	"0"	0.90	0.93	"0"	0.88	0.92
"*"	1.00	0.00	"*"	1.03	0.17	"*"	1.03	0.24
"#"	0.00	1.00	"#"	0.13	1.02	"#"	0.22	1.02

Example 4.12

Figure 4.31
Results after three epochs.

Figure 4.32
Results after 10 epochs.

Figure 4.33
Results after 40 epochs.

Weights after epoch 3:

"+"	0.36	0.57
"0"	0.89	0.92
"*"	1.05	0.26
"#"	0.27	1.00

Weights after epoch 10:

"+"	0.34	0.44
"0"	0.89	0.91
"*"	1.10	0.26
"#"	0.30	1.03

Weights after epoch 40:

"+"	0.30	0.31
"0"	0.92	0.93
"*"	1.11	0.26
"#"	0.27	1.09

Variations

- ❑ We now consider several improved LVQ algorithms, called LVQ2, LVQ2.1 and LVQ3.
- ❑ In the original LVQ algorithm, only the reference vector that is closest to the input vector is updated.
- ❑ The direction it is moved depends on whether the winning reference vector belongs to the same class as the input vector.
- ❑ In the improved algorithms, two vectors (the winner and a runner-up) learn if several conditions are satisfied.
- ❑ The idea is that if the input is approximately the same distance from both the winner and the runner-up, then each of them should learn.

LVQ2

- ❑ In the first modification, LVQ2, the conditions under which both vectors are modified are that:
 - ❑ 1. The winning unit and the runner-up (the next closest vector) represent different classes.
 - ❑ 2. The input vector belongs to the same class as the runner-up.
 - ❑ 3. The distances from the input vector to the winner and from the input vector to the runner-up are approximately equal. This condition is expressed in terms of a window, using the following notation:
 - ❑ x current input vector;
 - ❑ Y_c reference vector that is closest to x ;

LVQ2

- ❑ y_r reference vector that is next to closest to x (the runner-up);
- ❑ d_c distance from x to y_c ;
- ❑ d_r distance from x to y_r .
- ❑ To be used in updating the reference vectors, a window is defined as follows:
- ❑ The input vector x falls in the window if

$$\frac{d_c}{d_r} > 1 - \epsilon$$

$$\frac{d_r}{d_c} < 1 + \epsilon,$$

- ❑ where the value of ϵ depends on the number of training samples; a value of .35 is typical.

LVQ2

- ❑ In LVQ2, the vectors y_c , and y_r , are updated if the input vector x falls in the window, y_c , and y_r , belong to different classes, and x belongs to the same class as y_r .
- ❑ If these conditions are met, the closest reference vector and the runner up are updated:

$$y_c(t + 1) = y_c(t) - \alpha(t)[x(t) - y_c(t)];$$

$$y_r(t + 1) = y_r(t) + \alpha(t)[x(t) - y_r(t)].$$

LVQ2.1

- ❑ In the modification called LVQ2.1 Kohonen considers the two closest reference vectors, $yc1$ and $yc2$.
- ❑ The requirement for updating these vectors is that one of them, say, $yc1$, belongs to the correct class (for the current input vector x) and the other ($yc2$) does not belong to the same class as x .
- ❑ Unlike LVQ2, LVQ2.1 does not distinguish between whether the closest vector is the one representing the correct class or the incorrect class for the given input.

LVQ2.1

- ❑ As with LVQ2, it is also required that x fall in the window in order for an update to occur.
- ❑ The test for the window condition to be satisfied becomes
$$\min \left[\frac{d_{c1}}{d_{c2}}, \frac{d_{c2}}{d_{c1}} \right] > 1 - \epsilon$$
- ❑ And
$$\max \left[\frac{d_{c1}}{d_{c2}}, \frac{d_{c2}}{d_{c1}} \right] < 1 + \epsilon.$$
- ❑ The more complicated expressions result from the fact that we do not know whether x is closer to $yc1$ or to $yc2$.

LVQ2.1

- ❑ If these conditions are met, the reference vector that belongs to the same class as x is updated according to

$$y_{c1}(t + 1) = y_{c1}(t) + \alpha(t)[x(t) - y_{c1}(t)],$$

- ❑ and the reference vector that does not belong to the same class as x is updated according to

$$y_{c2}(t + 1) = y_{c2}(t) - \alpha(t)[x(t) - y_{c2}(t)].$$

LVQ2.1

- ❑ to learn as long as the input vector satisfies the window condition

$$\min \left[\frac{d_{c1}}{d_{c2}}, \frac{d_{c2}}{d_{c1}} \right] > (1 - \epsilon)(1 + \epsilon),$$

- ❑ where typical values of $\epsilon = 0.2$ are indicated. (Note that this window condition is also used for LVQ2 in Kohonen.)
- ❑ If one of the two closest vectors, $yc1$, belongs to the same class as the input vector x , and the other vector $yc2$ belongs to a different class, the weight updates are as for LVQ2.1.

LVQ3

- ❑ However, LVQ3 extends the training algorithm to provide for training if x , $yc1$, and $yc2$ belong to the same class.
- ❑ In this case, the weight updates are:
$$y_c(t + 1) = y_c(t) + \beta(t)[x(t) - y_c(t)]$$
- ❑ for both $yc1$ and $yc2$.
- ❑ The learning rate $\beta(t)$ is a multiple of the learning rate $\alpha(t)$ that is used if $yc1$, and $yc2$ belong to different classes.
- ❑ The appropriate multiplier is typically between 0.1 and 0.5, with smaller values corresponding to a narrower window.

LVQ3

- ❑ Symbolically,
- ❑ $\beta(t) = m \alpha(t)$ for $.1 < m < 0.5$.
- ❑ This modification to the learning process ensures that the weights (codebook vectors) continue to approximate the class distributions and prevents the codebook vectors from moving away from their optimal placement if learning continues.

Counterpropagation

- ❑ Counterpropagation networks are multilayer networks based on a combination of input, clustering, and output layers.
- ❑ Counterpropagation nets can be used to compress data, to approximate functions, or to associate patterns.
- ❑ A counterpropagation net approximates its training input vector pairs by adaptively constructing a look-up table.
- ❑ In this manner, a large number of training data points can be compressed to a more manageable number of look-up table entries.

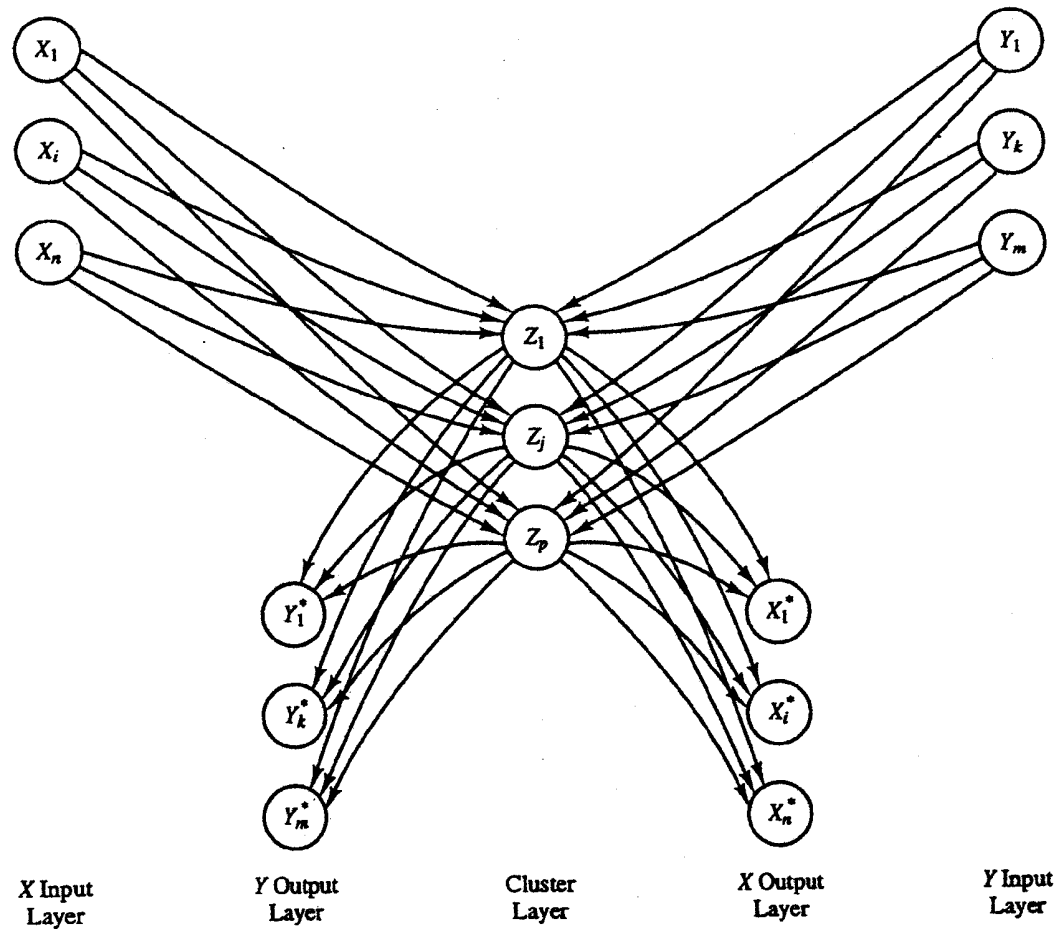
Counterpropagation

- ❑ Counterpropagation nets are trained in two stages.
- ❑ During the first stage, the input vectors are clustered based on either the dot product metric or the Euclidean norm metric.
- ❑ During the second stage of training, the weights from the cluster units to the output units are adapted to produce the desired response.
- ❑ There are two types of counterpropagation nets: full and forward only.

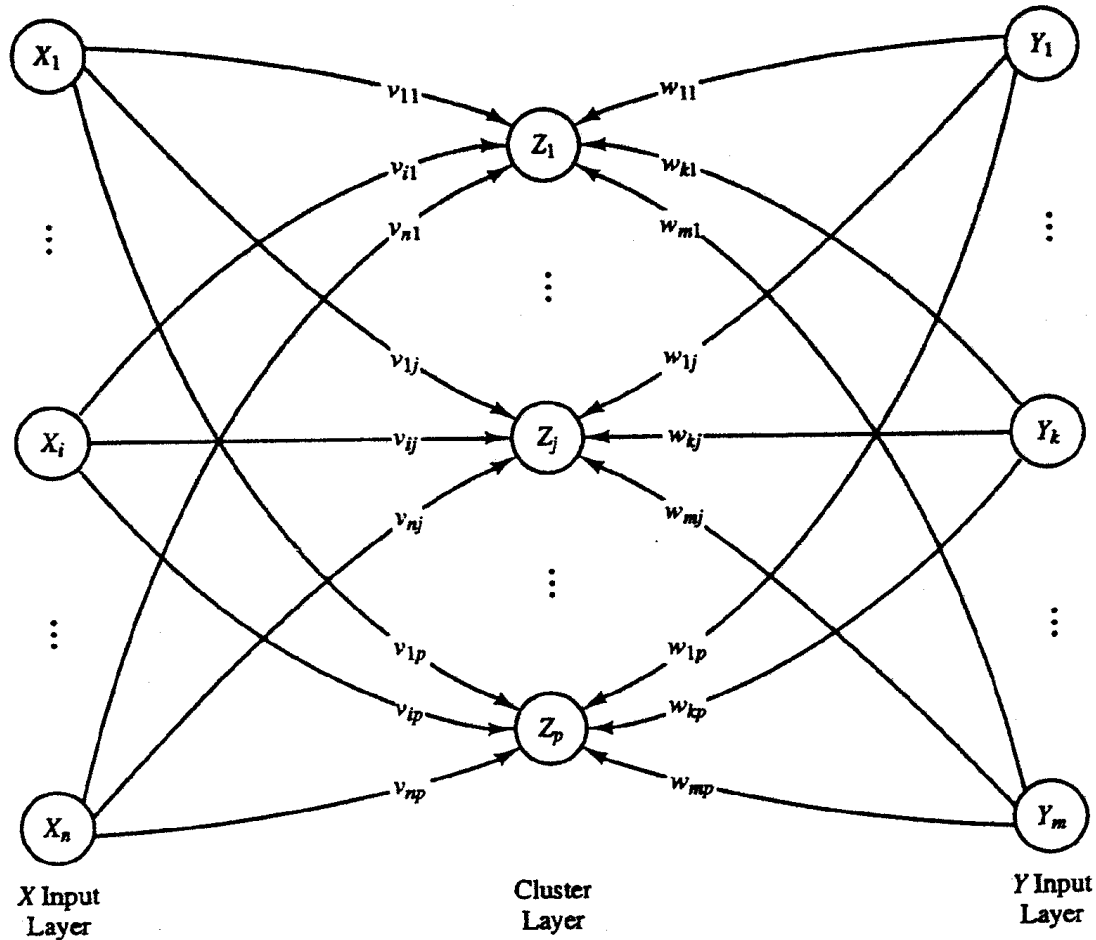
Full Counterpropagation

- ❑ Full counterpropagation was developed to provide an efficient method of representing a large number of vector pairs, $x:y$ by adaptively constructing a lookup table.
- ❑ It produces an approximation $x^*:y^*$ based on input of an x vector (with no information about the corresponding y vector), or input of a y vector only, or input of an $x:y$ pair, possibly with some distorted or missing elements in either or both vectors.
- ❑ Full counterpropagation uses the training vector pairs $x:y$ to form the clusters during the first phase of training.

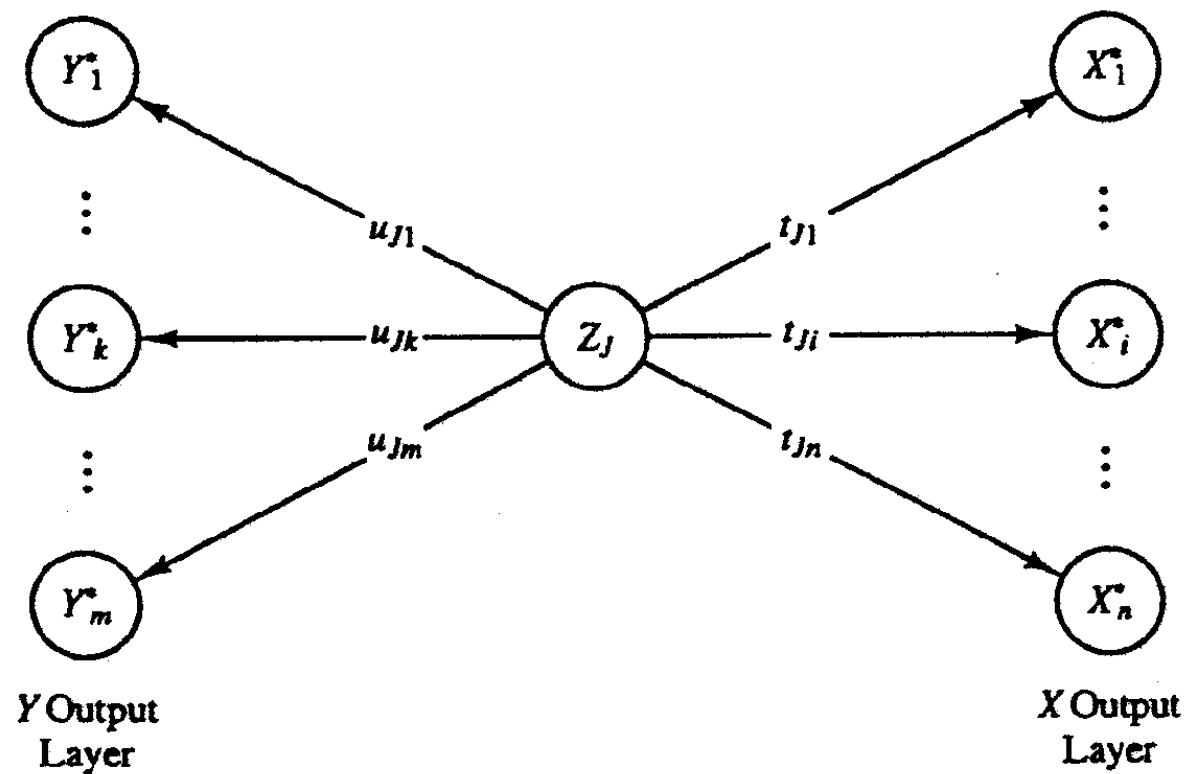
Full Counterpropagation



First phase of training



Second phase of training



Algorithm

- ❑ Training a counterpropagation network occurs in two phases.
- ❑ During the first phase, the units in the X input, cluster, and Y input layers are active.
- ❑ The units in the cluster layer compete; the interconnections are not shown.
- ❑ In the basic definition of counterpropagation, no topology is assumed for the cluster layer units; only the winning unit is allowed to learn.

Algorithm

- ❑ The learning rule for weight updates on the winning cluster unit is

$$v_{iJ}(\text{new}) = (1 - \alpha)v_{iJ}(\text{old}) + \alpha x_i, \quad i = 1, \dots, n;$$

$$w_{kJ}(\text{new}) = (1 - \beta)w_{kJ}(\text{old}) + \beta y_k, \quad k = 1, \dots, m.$$

- ❑ This is standard Kohonen learning, which consists of both the competition among the units and the weight updates for the winning unit.
- ❑ During the second phase of the algorithm, only unit J remains active in the cluster layer.
- ❑ The weights from the winning cluster unit J to the output units are adjusted so that the vector of activations of the units in the Y output layer, y^* , is an approximation to the input vector y ; x^* is an approximation to x .

Algorithm

- ❑ The weight updates for the units in the Y output and X output layers are

$$u_{Jk}(\text{new}) = (1 - a)u_{Jk}(\text{old}) + ay_k, \quad k = 1, \dots, m;$$

$$t_{Ji}(\text{new}) = (1 - b)t_{Ji}(\text{old}) + bx_i, \quad i = 1, \dots, n.$$

- ❑ This is known as Grossberg learning, which, as used here, is a special case of the more general outstar learning.
- ❑ Outstar learning occurs for all units in a particular layer; no competition among those units is assumed.
- ❑ However, the forms of the weight updates for Kohonen learning and Grossberg learning are closely related



Algorithm

- ❑ The weight updates for the units in the Y output and X output layers are

$$u_{Jk}(\text{new}) = (1 - a)u_{Jk}(\text{old}) + ay_k, \quad k = 1, \dots, m;$$

$$t_{Ji}(\text{new}) = (1 - b)t_{Ji}(\text{old}) + bx_i, \quad i = 1, \dots, n.$$

- ❑ This is known as **Grossberg learning**, which, as used here, is a special case of the more general outstar learning.

- ❑ Now, simple algebra gives

$$\begin{aligned} u_{Jk}(\text{new}) &= (1 - a)u_{Jk}(\text{old}) + ay_k \\ &= u_{Jk}(\text{old}) + a(y_k - u_{Jk}(\text{old})). \end{aligned}$$

- ❑ Thus, the weight change is simply the learning rate a times the error.



Algorithm

- ❑ x input training vector: $\mathbf{x} = (x_1, \dots, x_i, \dots, x_n)$.
- ❑ Y target output corresponding to input x :

$$\mathbf{y} = (y_1, \dots, y_k, \dots, y_m).$$

z_j	activation of cluster layer unit Z_j .
$\mathbf{x}^*$	computed approximation to vector $\mathbf{x}$.
$\mathbf{y}^*$	computed approximation to vector $\mathbf{y}$.
v_{ij}	weight from X input layer, unit X_i , to cluster layer, unit Z_j .
w_{kj}	weight from Y input layer, unit Y_k , to cluster layer, unit Z_j .
u_{jk}	weight from cluster layer, unit Z_j , to Y output layer, unit Y_k^* .
t_{ji}	weight from cluster layer, unit Z_j , to X output layer, unit X_i^* .
α, β	learning rates for weights into cluster layer (Kohonen learning).
a, b	learning rates for weights out from cluster layer (Grossberg learning).

Algorithm

- Step 0.* Initialize weights, learning rates, etc.
- Step 1.* While stopping condition for phase 1 training is false, do Steps 2–7.
- Step 2.* For each training input pair $\mathbf{x}:\mathbf{y}$, do Steps 3–5.
- Step 3.* Set X input layer activations to vector $\mathbf{x}$;
set Y input layer activations to vector $\mathbf{y}$.
- Step 4.* Find winning cluster unit; call its index J :
- Step 5.* Update weights for unit Z_J :

$$v_{iJ}(\text{new}) = (1 - \alpha)v_{iJ}(\text{old}) + \alpha x_i,$$

$$i = 1, \dots, n;$$

$$w_{kJ}(\text{new}) = (1 - \beta)w_{kJ}(\text{old}) + \beta y_k,$$

$$k = 1, \dots, m.$$

Algorithm

- Step 6.* Reduce learning rates α and β .
- Step 7.* Test stopping condition for phase 1 training.
- Step 8.* While stopping condition for phase 2 training is false, do Steps 9–15.
(*Note:* α and β are small, constant values during phase 2.)
- Step 9.* For each training input pair $x:y$, do Steps 10–13.
- Step 10.* Set X input layer activations to vector x ;
set Y input layer activations to vector y .
- Step 11.* Find winning cluster unit; call its index J .
- Step 12.* Update weights into unit Z_J :

$$v_{iJ}(\text{new}) = (1 - \alpha)v_{iJ}(\text{old}) + \alpha x_i,$$

$$i = 1, \dots, n;$$

$$w_{kJ}(\text{new}) = (1 - \beta)w_{kJ}(\text{old}) + \beta y_k.$$

$$k = 1, \dots, m.$$

Algorithm

Step 13. Update weights from unit Z_J to the output layers:

$$u_{Jk}(\text{new}) = (1 - a)u_{Jk}(\text{old}) + ay_k,$$

$$k = 1, \dots, m.$$

$$t_{Ji}(\text{new}) = (1 - b)t_{Ji}(\text{old}) + bx_i,$$

$$i = 1, \dots, n.$$

Step 14. Reduce learning rate.

Step 15. Test stopping condition for phase 2 training.

In Steps 4 and 11:

In case of a tie, take the unit with the smallest index.

Algorithm

- ❑ To use the dot product metric, find the cluster unit Z_j with the largest net input:

$$Z_{in_j} = \sum_i x_i v_{ij} + \sum_k y_k w_{kj}$$

- ❑ The weight vectors and input vectors should be normalized to use the dot product metric.
- ❑ To use the Euclidean distance metric, find the cluster unit Z_j , the square of whose distance from the input vectors is smallest:

$$D_j = \sum_i (x_i - v_{ij})^2 + \sum_k (y_k - w_{kj})^2.$$

Application

- ❑ After training, a counterpropagation neural net can be used to find approximations x^* and y^* to the input, output vector pair x and y .
- ❑ Hecht-Nielsen refers to this process as accretion, as opposed to interpolation between known values of a function.
- ❑ The application procedure for counterpropagation is as follows:

Application

- Step 0.* Initialize weights.
- Step 1.* For each input pair $x:y$, do Steps 2–4.
- Step 2.* Set X input layer activations to vector x ;
set Y input layer activations to vector y ;
- Step 3.* Find the cluster unit Z_J that is closest to the input pair.
- Step 4.* Compute approximations to x and y :

$$x_i^* = t_{ji},$$

$$y_k^* = u_{jk}.$$

Application

- ❑ The net can also be used in an interpolation mode; in this case, several units are allowed to be active in the cluster layer.
- ❑ The interpolated approximations to x and y are then:

$$x_i^* = \sum_j z_j t_{ji},$$

$$y_k^* = \sum_j z_j u_{jk}.$$

- ❑ For testing with only an x vector for input (i.e., there is no information about the corresponding y), it may be preferable to find the winning unit J based on comparing only the x vector and the first n components of the weight vector for each cluster layer unit.

Example 4.14

- ❑ A full counterpropagation net for the function $y = 1/x$.
- ❑ Suppose we have 10 cluster units (in the Kohonen layer); there is 1 X input layer unit, 1 Y input layer unit, 1 X output layer unit, and 1 Y output layer unit.
- ❑ Suppose further that we have a large number of training points (perhaps 1,000), with x values between 0.1 and 10.0 and the corresponding y values given by $y = 1/x$.
- ❑ The training input points, which are uniformly distributed along the curve, are presented in random order.
- ❑ If our initial weights (on the cluster units) are chosen appropriately, then after the first phase of training, the clusters units will be uniformly distributed along the curve.

Example 4.14

- ❑ The first weight for each cluster unit is the weight from the X input unit, the second weight the weight from the Y input unit.
- ❑ We have:

CLUSTER UNIT	v	w
Z_1	0.11	9.0
Z_2	0.14	7.0
Z_3	0.20	5.0
Z_4	0.30	3.3
Z_5	0.6	1.6
Z_6	1.6	0.6
Z_7	3.3	0.30
Z_8	5.0	0.20
Z_9	7.0	0.14
Z_{10}	9.0	0.11

Example 4.14

- ❑ After the second phase of training, the weights to the output units will be approximately the same as the weights into the cluster units.
- ❑ we can use this net to obtain the approximate value of y for $x = 0.12$ as follows:
- ❑ Step 0. Initialize weights.
- ❑ Step 1 . For the input $x=0.12, y=0.0$, doSteps2-4.
- ❑ Step 2. Set X input layer activations to vector x ; set Y input layer activations to vector y ;

Example 4.14

- Step 3. Find the index J of the winning cluster unit; the squares of the distances from the input to each of the cluster units are:

$$D_1 = (0.12 - 0.11)^2 + (0.00 - 9.00)^2 = 81$$

$$D_2 = (0.12 - 0.14)^2 + (0.00 - 7.00)^2 = 49$$

$$D_3 = (0.12 - 0.20)^2 + (0.00 - 5.00)^2 = 25$$

$$D_4 = (0.12 - 0.30)^2 + (0.00 - 3.30)^2 = 11$$

$$D_5 = (0.12 - 0.60)^2 + (0.00 - 1.60)^2 = 2.8$$

$$D_6 = (0.12 - 1.60)^2 + (0.00 - 0.60)^2 = 2.6$$

$$D_7 = (0.12 - 3.30)^2 + (0.00 - 0.30)^2 = 10.2$$

$$D_8 = (0.12 - 5.00)^2 + (0.00 - 0.20)^2 = 23.9$$

$$D_9 = (0.12 - 7.00)^2 + (0.00 - 0.14)^2 = 47.4$$

$$D_{10} = (0.12 - 9.00)^2 + (0.00 - 0.11)^2 = 78.9$$

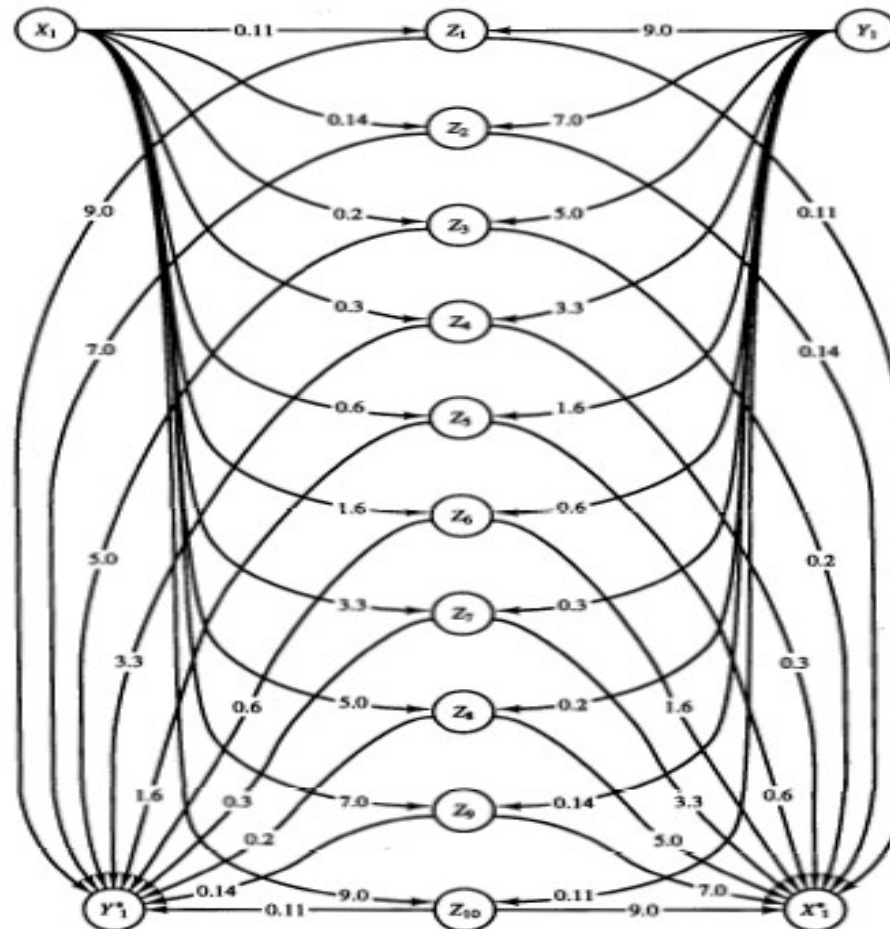
Example 4.14

- Step 4. Compute approximations

$$\mathbf{x}^* = t_J = 1.6$$

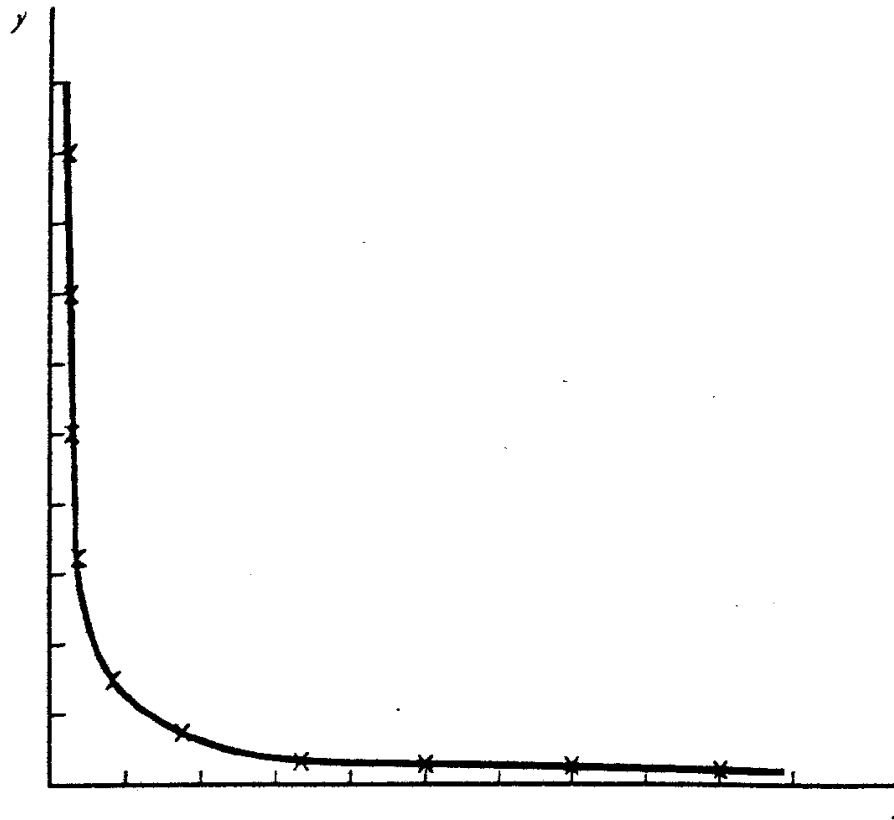
$$\mathbf{y}^* = u_J = 0.6.$$

Example 4.14



Example 4.14

- position of cluster units



Example 4.14

- ❑ Clearly, this is not really the approximation we wish to find.
- ❑ Since we only have information about the x input, we should use the earlier mentioned modification to the application procedure.
- ❑ Thus, if we base our search for the winning cluster unit on distance from the x input to the corresponding weight for each cluster unit, we find the following in Steps 3 and 4:

Example 4.14

- Step 3. Find the index J of the winning cluster unit; the squares of the distances from the input to each of the cluster units are:

$$D_1 = (0.12 - 0.11)^2 = 0.0001$$

$$D_2 = (0.12 - 0.14)^2 = 0.0004$$

$$D_3 = (0.12 - 0.20)^2 = 0.064$$

$$D_4 = (0.12 - 0.30)^2 = 0.032$$

$$D_5 = (0.12 - 0.60)^2 = 0.23$$

$$D_6 = (0.12 - 1.60)^2 = 2.2$$

$$D_7 = (0.12 - 3.30)^2 = 10.1$$

$$D_8 = (0.12 - 5.00)^2 = 23.8$$

$$D_9 = (0.12 - 7.00)^2 = 47.3$$

$$D_{10} = (0.12 - 9.00)^2 = 81$$

Example 4.14

- Thus, based on the input from x only, the closest cluster unit is $J = 1$.

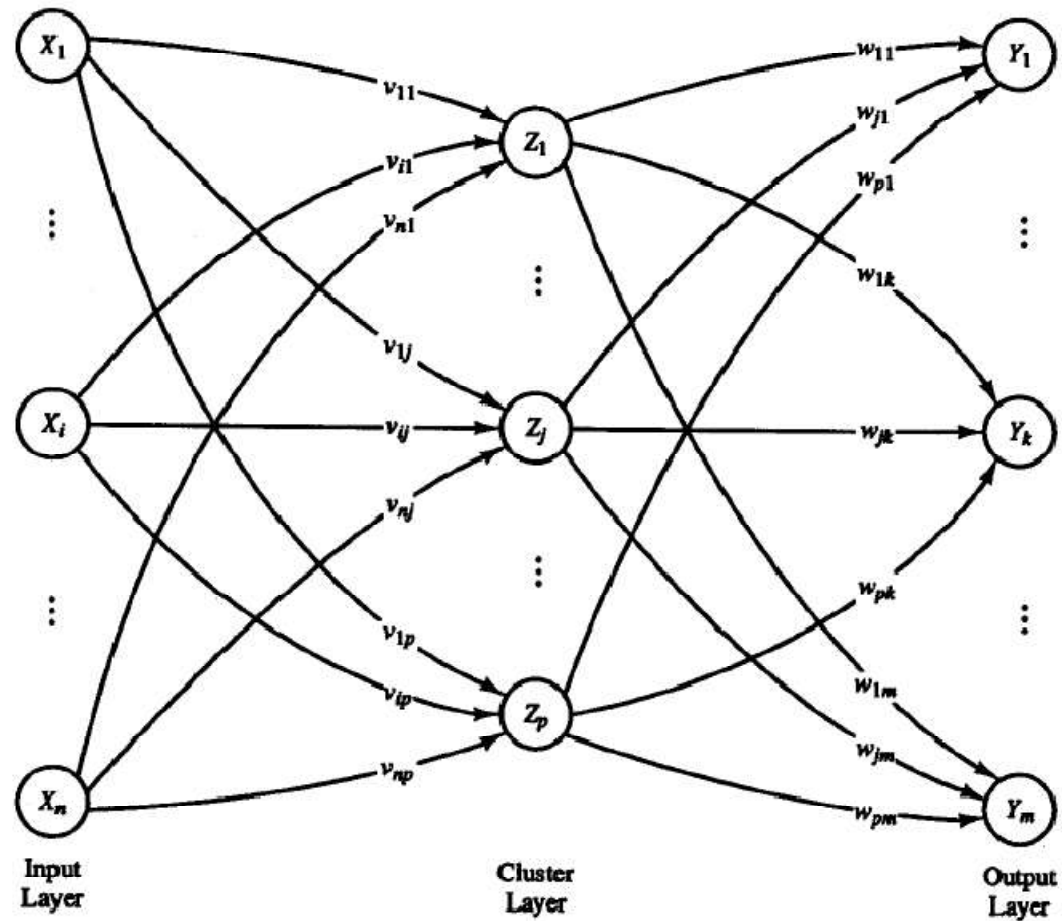
$$\mathbf{x}^* = \mathbf{t}_J = 0.11,$$

$$y^* = u_J = 9.00.$$

Forward-Only

- ❑ Forward-only counterpropagation nets are a simplified version of the full counterpropagation nets.
- ❑ Forward-only nets are intended to approximate a function $y = f(x)$ that is not necessarily invertible; that is, forward-only counterpropagation nets may be used if the mapping from x to y is well defined, but the mapping from y to x is not.
- ❑ Forward-only counterpropagation differs from full counterpropagation in using only the x vectors to form the clusters on the Kohonen units during the first stage of training.

Forward-Only



Algorithm

- ❑ The training procedure for the forward-only counterpropagation net consists of several steps, as indicated in the algorithm that follows.
- ❑ First, an input vector is presented to the input units.
- ❑ The units in the cluster layer compete (winner take all) for the right to learn the input vector.
- ❑ After the entire set of training vectors has been presented, the learning rate is reduced and the vectors are presented again; this continues through several iterations.

Algorithm

- ❑ After the weights from the input layer to the cluster layer have been trained (the learning rate has been reduced to a small value), the weights from the cluster layer to the output layer are trained.
- ❑ Now, as each training input vector is presented to the input layer, the associated target vector is presented to the output layer.
- ❑ The winning cluster unit (call it J) sends a signal of 1 to the output layer.
- ❑ Each output unit k has a computed input signal W_{Jk} and target value Y_k .

Algorithm

- ❑ Using the difference between these values, the weights between the winning cluster unit and the output layer are updated.
- ❑ The learning rule for these weights is similar to the learning rule for the weights from the input units to the cluster units
- ❑ The nomenclature used is as follows:
- ❑ α, a learning rate parameters:
$$.5 < \alpha < .8,$$
$$0 < a < 1$$

Algorithm

- Step 0.* Initialize weights, learning rates, etc.
- Step 1.* While stopping condition for phase 1 is false, do Steps 2–7.
- Step 2.* For each training input $\mathbf{x}$, do Steps 3–5.
- Step 3.* Set X input layer activations to vector $\mathbf{x}$.
- Step 4.* Find winning cluster unit; call its index J .
- Step 5.* Update weights for unit Z_J :
- $$v_{iJ}(\text{new}) = (1 - \alpha)v_{iJ}(\text{old}) + \alpha x_i,$$
- $$i = 1, \dots, n.$$

Algorithm

- Step 6.* Reduce learning rate α .
- Step 7.* Test stopping condition for phase 1 training.
- Step 8.* While stopping condition for phase 2 training is false, do Steps 9–15.
(*Note:* α is a small, constant value during phase 2.)
- Step 9.* For each training input pair $\mathbf{x}:\mathbf{y}$, do Steps 10–13.
- Step 10.* Set X input layer activations to vector $\mathbf{x}$;
set Y output layer activations to vector $\mathbf{y}$.
- Step 11.* Find the winning cluster unit; call its index J .
- Step 12.* Update weights into unit Z_J (α is small):
$$v_{iJ}(\text{new}) = (1 - \alpha)v_{iJ}(\text{old}) + \alpha x_i,$$
$$i = 1, \dots, n.$$
- Step 13.* Update weights from unit Z_J to the output units:
$$w_{Jk}(\text{new}) = (1 - a)w_{Jk}(\text{old}) + ay_k,$$
$$k = 1, \dots, m.$$
- Step 14.* Reduce learning rate a .
- Step 15.* Test stopping condition for phase 2 training.

Applications

- ❑ The application procedure for forward-only counterpropagation is:
- ❑ Step 0. Initialize weights (by training as in previous subsection).
- ❑ Step 1. Present input vector x .
- ❑ Step 2. Find unit J closest to vector x .
- ❑ Step 3. Set activations of output units:

$$y_k = w_{Jk}.$$

- ❑ A forward-only counterpropagation net can also be used in an "interpolation" mode.

Applications

- ❑ In this case, more than one Kohonen unit has a nonzero activation with

$$\sum_j z_j = 1.$$

- ❑ The activation of the output units is then given by

$$y_k = \sum_j z_j w_{jk}.$$

- ❑ Again, accuracy is increased by using the interpolation mode.

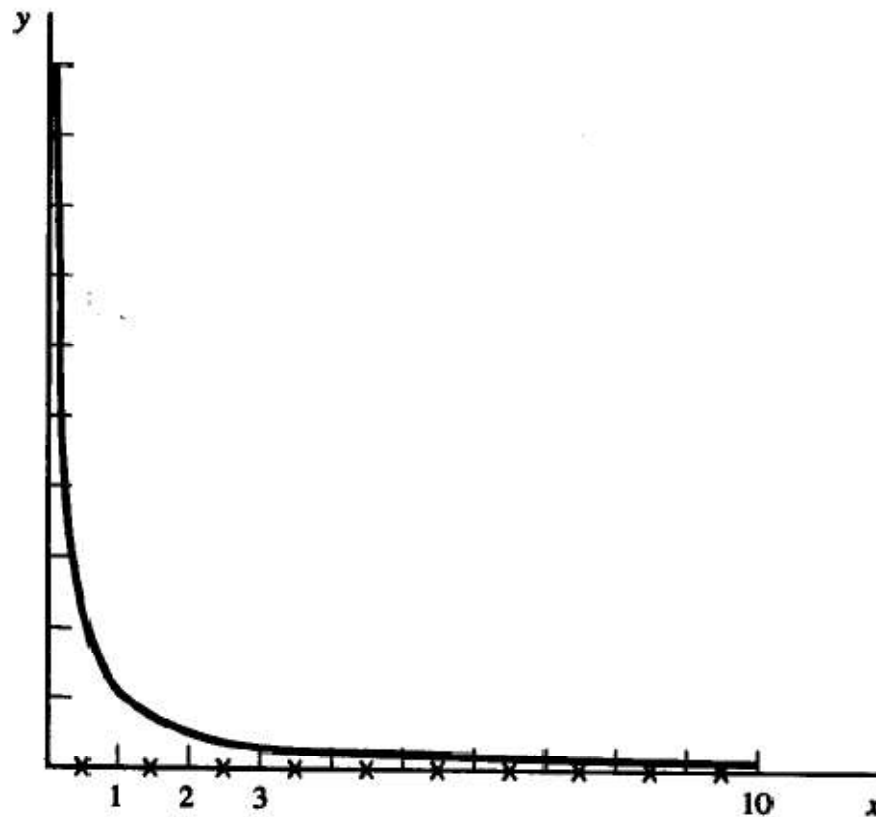
Example 4.15

- ❑ A forward-only counterpropagation net for the function $y = 1/x$.
- ❑ In this example, we consider the performance of a forward-only counterpropagation net to form a look-up table for the function $y = 1/x$ on the interval $[0.1, 10.0]$.
- ❑ Suppose we have 10 cluster units (in the cluster layer); there is 1 X input layer unit and 1 Y output layer unit.
- ❑ Suppose further that we have a large number of training points (the x values for our function) uniformly distributed between 0.1 and 10.0 and presented in a random order.

Example 4.15

- ❑ If we use a linear structure on the cluster units, the weights (from the input unit to the 10 cluster units) will be approximately 0.5, 1.5, 2.5, 3.5, . . . , 9.5 after the first phase of training.
- ❑ After the second phase of training, the weights to the Y output units will be approximately 5.5, 0.75, 0.4, . . . , 0.1.
- ❑ Thus, the approximations to the function values will be much more accurate for large values of x than for small values.

Example 4.15



Example 4.15

- ❑ Comparing these results with those of Example 4.14 (for full counterpropagation), we see that even if the net is intended only for approximating the mapping from x to y , the full counterpropagation net may distribute the cluster units in a manner that produces more accurate approximations over the entire range of input values